

烟幕干扰弹的投放策略问题

摘要

本研究针对寻找烟幕干扰弹的最佳投放策略的问题，通过建立三维数学模型和运动学模型，使用粒子群等智能优化算法等进行优化，提出了解决类似问题的模型。

针对问题一，本文运用**空间解析几何理论**建立了严谨的三维坐标系统。为解决遮蔽判定的复杂性问题，设计了一种**基于蒙特卡罗方法的混合判定算法**，通过在目标区域内进行多点随机采样，结合**统计覆盖率分析实现遮蔽效果**的定量化评价。经过数值仿真和参数优化，获得最优时间解区间为 $[1.392, 1.429]$ ，遮蔽时间为 **1.409s**

针对问题二，本文在问题一所构建理论框架的基础上，以无人机飞行角度、飞行时间、烟雾弹投放时间以及爆炸时间为决策变量，构建四维参数空间，基于题目给定的物理约束条件和边界条件，将问题转化为**约束优化问题**。并采用**粒子群优化算法**对该问题进行数值求解，通过迭代寻优获得问题的近似最优解为 **4.6 秒**

针对问题三，本文基于问题二的研究基础，将待决策变量提升至八个，同时采用**时间轴并集算法**计算三枚烟雾弹的联合遮蔽时长，避免重叠区域的重复统计。通过改进粒子群算法的**序列优化策略**，提升算法收敛性能，最终获得 **6.1 秒**的最优解。

针对问题四，继续将待决策变量扩展至 12 个，本文基于问题二的理论框架，在使用粒子群优化之前首先使用**粗网格搜索**确定较优的参数组合作为起点，减少了算力需求，并设计了**多准则评价体系**，采用综合评分函数对候选解进行量化排序，进行有效识别和筛选高质量解；同时引入**多样性保持策略和重启机制**，显著增强了算法跳出局部最优解的能力。通过上述综合优化策略，成功解决了高维空间中的**局部收敛**问题，最终获得全局最优遮蔽时长为 **8.312 秒**。

针对问题五中五架无人机协同拦截三枚导弹、超 40 维的复杂优化问题，本文提出了基于层次分解的**分类决策策略**。首先将原始的超高维优化问题分解为**无人机-导弹任务分配**子问题和各分组**内部参数优化**子问题，再采用枚举法对所有可能的任务分配方案进行**遍历分析**，确定最优分配策略。最后针对每种分配方案，运用**粒子群优化算法**对相应的参数子空间进行精细化优化，显著降低了问题的计算复杂度，同时保证了解的质量。经过综合优化分析，获得理论最优遮蔽时长约为 20.030 秒。

关键词：粒子群优化算法 决策树 优化算法 网格遍历 Sigmoid 时间编码 Nelder - Mead 精修

一、问题重述

1.1 问题背景

随着精确制导技术的快速发展，其智能化和抗干扰能力不断提升，实现了常规弹药集群攻击模式向制导弹药精确打击模式的转变。为有效对抗精确制导武器，烟幕干扰技术快速发展。其中，通过对烟幕干扰弹的定点精确抛撒可以有效对导弹产生干扰。运用无人机可以完成烟幕干扰弹的定点精确抛撒问题，无人机的飞行方向、飞行速度和烟幕干扰弹的投放点等因素影响到烟幕干扰弹对目标的遮蔽效果。因此需要考虑烟幕干扰弹的投放策略，使其对目标的有效遮蔽时间尽可能长。

1.2 问题重述

针对导弹对真目标的威胁，现采用无人机对烟幕干扰弹进行抛撒。干扰弹能通过形成烟幕或气溶胶云团对空域形成遮蔽，从而对来袭导弹造成视线干扰。当导弹朝着为掩护真目标设置的假目标方向移动时，通过无人机投放烟幕干扰弹能尽量避免导弹发现真目标。

以假目标为原点，已知真目标、各导弹和各无人机的位置信息，真目标为半径为 7m、高为 10m 的圆柱体。现需合理规划无人机的飞行方向、飞行速度，确定烟幕干扰弹的抛撒地点和爆炸时刻，使得多枚烟幕干扰弹能尽可能长时间遮蔽真目标。

问题一 无人机 FY1 以 120m/s 的速度等高匀速直线向假目标方向飞行（俯冲），接到任务 1.5s 后投放一枚烟幕干扰弹，将在 3.6s 后起爆。爆炸后形成球状烟幕云团并以 3m/s 的速度匀速下沉，云团中心 10m 范围内的烟幕浓度可在起爆后 20s 内对目标有效遮蔽。现有导弹 M1 以 300m/s 朝假目标方向飞行，求烟幕干扰弹对 M1 的有效遮蔽时长。

问题二 在 FY1 飞行信息未知的情况下，FY1 投放一枚烟幕弹。需要确定 FY1 的飞行方向和速度、烟幕干扰弹投放点和起爆点，使其对 M1 的干扰时间最长。

问题三 此问题将问题二拓展成 FY1 投放三枚烟幕弹，其中每枚烟幕弹的投放间

隔至少为 1s，需要给出无人机的投放策略，使得对 M1 的干扰时间最长。将结果保存到文件 result1.xlsx 中。

问题四 三架无人机 FY1、FY2 和 FY3 各投一枚烟幕干扰弹，实施对 M1 的干扰，需要给出烟幕干扰弹的投放策略使得对真目标的有效遮蔽时长最长，将结果保存到 result2.xlsx 中。

问题五 在问题四的基础上，利用五架无人机，每架无人机可投放三枚烟幕干扰弹，且间隔时长依然是 1s。实施对导弹 M1、M2、M3 的干扰，要求出干扰时间最长的策略并将结果存到 result3.xlsx 中。

二、问题分析

问题一 要求出投放烟幕干扰弹后对目标的有效遮蔽时长。可用题给条件建立所有物体的精确几何与运动模型。球状烟幕云团可视作半径为 10m 的球体，在真目标表面生成密集的点云，对于每个点，连接此点与 M1 形成空间直线，当球心到直线的距离小于或等于 10 时，可认定该点被烟幕遮挡。对于判定是否遮蔽真目标的问题，可用剪影覆盖率 coverage 来判别：设定阈值 α ，当

$\frac{\text{被遮挡的采样点数}}{\text{采样点总数}} = \text{coverage} > \alpha$ 时，可判定此时真目标被遮挡。从 $t = 0$ 开始，

以微小步长 dt 推进时间，在每个时间 t 更新导弹和烟幕干扰弹的位置，并计算剪影覆盖率，记录所有 $\text{coverage} > \alpha$ 的时间区间，将所有的时长加起来，得到总的遮蔽时间。

问题二 以第一问的动态仿真模型为基础，建立优化模型。该模型将无人机 FY1 的航向角、飞行速度、烟幕弹投放时刻以及引信延迟为决策变量，以最大化总遮蔽时长为优化目标。目标函数通过调用第一问建立的时间步进仿真程序实现。由于目标函数涉及复杂的动态几何计算和时间积分，因此采用粒子群优化算法（PSO）进行求解，通过智能优化算法在约束空间内寻找最优参数组合。

问题三 仍然只涉及 FY1，但它需要连续投放 3 枚烟幕干扰弹。由于不同烟幕云团的遮蔽时间可以相互衔接，这一问的重点在于如何合理安排投放时机和位置，使得 M1 在飞行过程中持续受到遮蔽，从而延长总的有效遮蔽时段。

问题四 情景扩展到多架无人机的协同。FY1、FY2 和 FY3 各投放一枚干扰弹，问

题的关键变成如何利用三架无人机的初始位置差异和机动自由度，通过合理设计各自的投放策略来实现对导弹 M1 的联合遮蔽。这要求我们从单机优化转向多机协同，考虑全局配合而不是单个个体。

问题五 针对问题 5 是整个题目的综合提升：5 架无人机、最多 15 枚烟幕干扰弹，要同时应对 3 枚导弹 M1、M2、M3。这不仅需要在空间和时间上合理分配无人机与弹药资源，还要协调多目标的干扰效果，保证三条导弹路径在接近真目标时尽可能处于遮蔽之中。这一问本质上是一个大规模组合优化与协同调度问题，也是对前面四问的综合应用与扩展。

三、模型假设

1. 导弹在飞行过程中视为质点，且始终按初始航线飞行，不会产生临时改变飞行轨迹的行为。
2. 本题中不考虑空气阻力、风速等因素的影响，烟幕干扰弹投下后爆炸前只受重力作用，爆炸后匀速下落。
3. 无人机在接到任务时可瞬时调整一次航向，随后以 70~140 m/s 的速度作等高度、匀速直线飞行，其速度和方向一旦确定不再改变。多架无人机不会碰撞
4. 由于导弹与地面距离较远，当导弹仅观测到真目标边界时无法识别真目标，只有当真目标的露出体积达到一定比例是才能被导弹识别。
5. 烟幕云团起爆后瞬时形成球形，不考虑烟雾弹烟雾的扩散时间，有效遮蔽区域为云团中心半径 10 m 的范围，该遮蔽效果持续 20 秒，到时立即失效。

四、符号说明

符号	说明	单位	属性
θ_i	无人机 i 飞行方向	度	决策变量
v_i	无人机 i 飞行速度	米/秒	决策变量
t	从零时刻起的时间	秒	变量
$P_{FY i}$	无人机 i 初始位置	/	常量

$t_{drop}(i,j)$	无人机 i 第 j 枚烟幕弹投放时	秒	决策变量
$P_{drop}(i,j)$	无人机 i 第 j 枚烟幕弹投放点	/	变量
$t_{boom}(i,j)$	无人机 i 第 j 枚烟幕弹起爆时	秒	决策变量
$C_{0(i,j)}$	无人机 i 第 j 枚烟幕弹起爆点	/	变量
$C_{(i,j)}(t)$	无人机 i 第 j 枚烟幕弹云团中心位置	/	变量

五、模型的建立和求解

5.1 问题一模型的建立与求解

5.1.1 模型的建立

(1) 导弹运动学模型的建立

已知 M1 的初始坐标 $P_M(0) = (20000, 0, 2000)$, 由于导弹的飞行方向指向假目标 $(0, 0, 0)$, 速度方向的单位向量为 $\frac{(-10, 0, -1)}{\sqrt{101}}$, 则飞行速度为 $v_M = 300 \frac{(-10, 0, -1)}{\sqrt{101}} m/s$ 。M1 的位移随时间变化模型为 $P_M(t) = P_M(0) + v_M t$

(2) 无人机和烟雾弹投放前运动学模型的建立

已知 FY1 的初始坐标 $P_{FY1}(0) = (17800, 0, 1800)$, 由于导弹的飞行方向指向假目标 $(0, 0, 0)$, 速度方向的单位向量为 $\frac{(-178, 0, -18)}{\|((-178, 0, -18))\|}$, 则飞行速度为 $v_{FY1} = 120 \frac{(-178, 0, -18)}{\|((-178, 0, -18))\|} m/s$ 。FY1 的位移随时间变化模型为 $P_{FY1}(t) = P_{FY1}(0) + v_{FY1} t$ 。

(3) 烟雾物理模型和运动学模型的建立

烟雾弹投放前的运动学模型与无人机的运动模型完全一致, 则烟幕干扰弹的运动模型为 $P_{Y1}(t) = P_{Y1}(0) + v_{Y1} t (0 \leq t \leq 1.5s)$ 。

烟雾弹投放后、爆炸前, 相对无人机做自由落体运动, 根据物理学基本原理,

$v_{Y2} = v_{FY1} + v_{\text{相对}}$, $v_{\text{相对}} = gt$, 代入可得 $P_{Y2}(t) = P_{Y1}(t1) + v_{Y1} t +$

$$1/2gt^2 \ (1.5s < t \leq 5.1s)$$

烟幕干扰弹爆炸后形成球状烟幕云团，烟幕云团以 $v_{Y3}=3m/s$ 的速度匀速下沉，

$$\text{此时 } P_{Y3}(t) = P_{Y2}(t_2) + v_{Y3} * t \ (t > 5.1s)$$

(4) 目标的数学模型建立

$$x^2 + (y - 200)^2 \leq 7^2, 0 \leq z \leq 10$$

目标的侧面可以描述为：

$$\begin{cases} x(\theta) = 7\cos\theta, \\ y(\theta) = 200 + 7\sin\theta, \theta \in [0, 2\pi) \\ z \in [0, 10], \end{cases}$$

目标的顶面可以描述为：

$$z = 10, x^2 + (y - 200)^2 \leq 49$$

(5) 圆柱体表面点云模型建立

基于假设四，可以在圆柱体表面选取一系列的离散的采样点，通过判断离散点的被遮蔽情况从而判断圆柱体是否被遮蔽。

侧面点集的生成

将圆周均匀分成 24 份，用弧度制表示圆周上的各个点的位置，记为 φ ；高度均匀分成 8 份，记为 h。通过遍历可得到 192 组 (φ, h) 组合。通过圆的参数方程可将每个 (φ, h) 组合转换为三维坐标 (x, y, z)

$$\begin{cases} x = r\cos(\varphi) \\ y = 200 + r\sin(\varphi) \\ z = h \end{cases}$$

顶面与底面点集的生成

首先确定在半径方向上取点个数，再将半径均匀分布形成集合 rs。以在 rs 中的 r' 为半径分别在顶面与底面上画圆。从 rs 中取出一个半径 r' ，遍历角度集合中的所有 θ ，生成 (r', θ) 组合，再通过极坐标转换得到三维直角坐标公式：

顶面

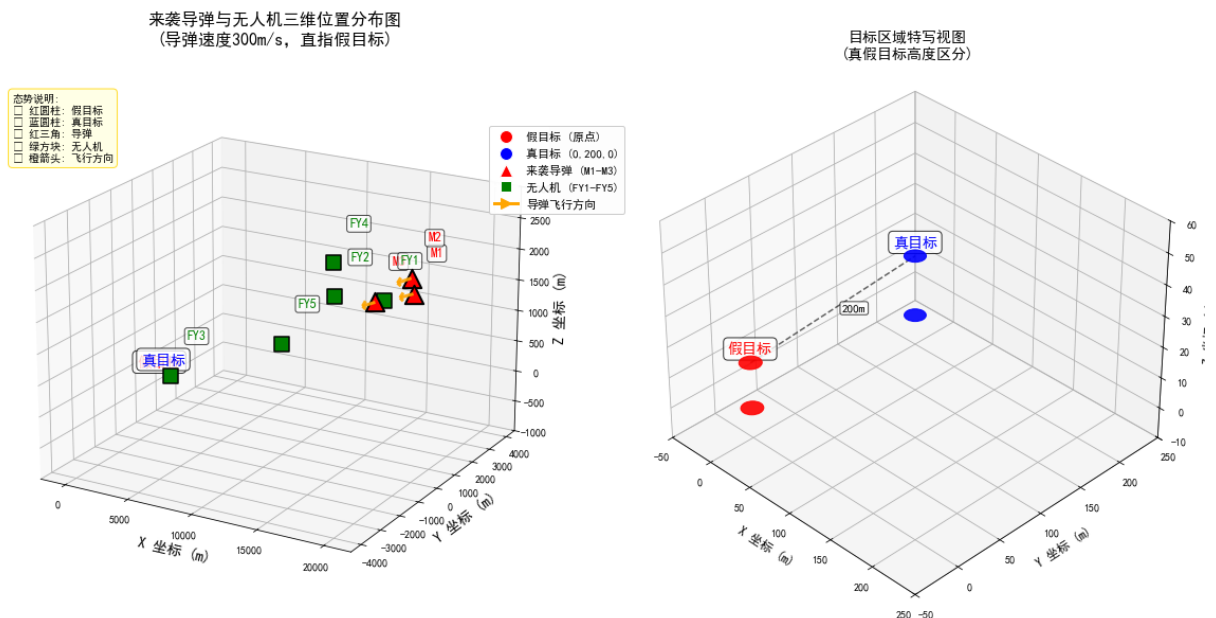
$$\begin{cases} x = r' \cos(\varphi) \\ y = 200 + r' \sin(\varphi) \\ z = 10 \end{cases}$$

底面

$$\begin{cases} x = r' \cos(\varphi) \\ y = 200 + r' \sin(\varphi) \\ z = 0 \end{cases}$$

这样就建立了一个圆柱体的离散点模型，便于后续求解

下面的可视化图便于理解：分别展示了导弹与无人机的位置关系和真假目标的位置关系图



(6) 遮蔽判断的几何模型建立

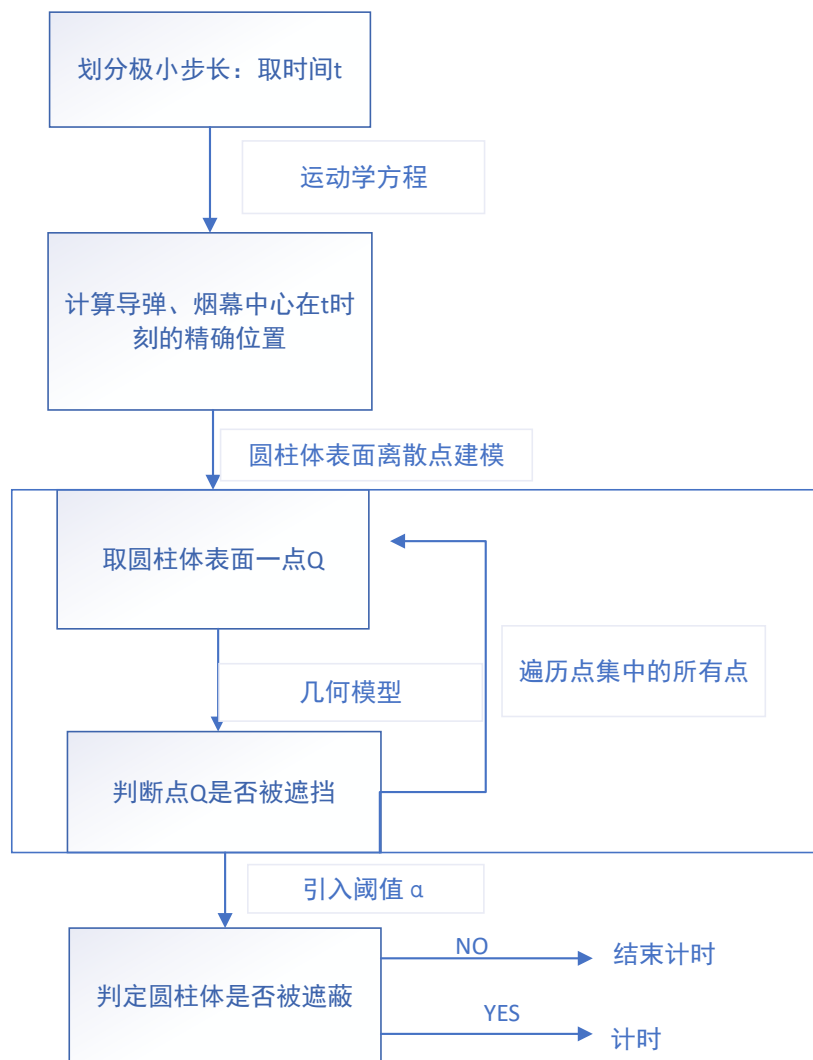
遮蔽判断条件是我们建立模型的关键。以上述建立的圆柱体表面点云模型为前置条件，以点集中的一点 A 为例，连接 A 与导弹 M1（可视为质点），其连线可视作空间直线。球状烟幕云团可简化看成半径为 10m 的球体。由于光沿直线传播，当光所在的直线与球体相切时刚好遮挡目标点 A。所以当球状烟幕云团中心到直线的距离小于其半径，即可认为云团对点 A 产生有效遮蔽。

针对遮蔽的讨论，我们有三种解决方案。方案 1: 把真目标抽象成一个点，直接导弹与目标点连线来判断，方案 2: 考虑到圆形不好建模，我们采取外切该

真目标圆柱的长方体，选取有几何特征的长方体点，如顶点，棱中点等。若该全部连线到烟雾弹球心到距离小于半径，认为遮挡。方案三：考虑到方案 2 判断条件的严苛，我们参考了现代解决这样问题的方法，是采取设置一个阈值，直接在圆柱体上随机取点（8-12 以上），若这些连线被遮挡的部分占全部连线的部分超过阈值，则判定遮挡。

经过讨论，我们认为方案三最合理，原因是它是最结合实际的方案，而且我们通过调节覆盖率阈值参数，可以清晰的看到有效遮挡时间的变化，加深了我们对问题的理解。

5.1.2 模型的求解



(1) 几何模型求解

选中圆柱形上点云集合中的一点 Q，以 5.1.1 中遮蔽判断的几何模型为基础，设导弹 M1 为点 M，球状烟幕云团的球心为 C。可通过投影的方法将向量 MC 投影到向量 MQ 上，设垂足为 H。定义 s 为垂足 H 在线段 MQ 上的相对位置：

$$s = \frac{\overrightarrow{MC} \cdot \overrightarrow{MQ}}{\|\overrightarrow{MQ}\|^2}$$

当 $0 < s < 1$ 时，H 在线段 MQ 上，此时为球体到直线的最短距离，如果 C 到直线的距离 d 小于球体半径 10m，则判断 Q 受到了遮挡：

$$d = \sqrt{(C_x - closest_x)^2 + (C_y - closest_y)^2 + (C_z - closest_z)^2}$$

(2) 烟幕云团是否有效遮蔽圆柱体的判断模型

定义阈值 α ，剪影覆盖率 coverage：

coverage = 点云中被有效遮挡的点 / 点云总数

α 为导弹成功识别出目标体时，未被观测体积占总体积的临界值。此处 $\alpha = 0.8$

当 $\alpha < \text{coverage}$ 的情形下，可认为真目标被有效遮蔽

(3) 求解遮蔽时间

a. 确定检查的时间段：

以警戒雷达发现导弹来袭为初始时刻， $t=0$ ；结束时间应为球状烟幕云团失效时刻与导弹击中假目标的时刻的最小值

烟幕云团失效时刻为 $T_{\text{failure}} = t_1 + t_2 + t_{\text{valid}} = 25.1\text{s}$ ；

导弹击中假目标的时刻 $T_{\text{hit}} = \frac{\|(20000, 0, 2000)\|}{v_M} = 66.999\text{m/s}$

故检查时间段应为 $t \in (0, T_{\text{failure}})$

b. 划分极小步长：

将连续时间划分成间隔为 dt 的离散时间点，可以认为近似地捕捉到连续过程

中的每个动态瞬间

取 $dt=0.004s$ ，则 $t=(0, 0.004, 0.008, \dots, 25.100)$

c. 遍历检查:

对于每个时间 t ，可通过运动学方程求解出导弹的精确位置 $P_M(t)$ ，和烟幕中心 C 的精确位置 $P_Y(t)$ ，其中

$$P_Y = \begin{cases} P_{Y1}, 0 \leq t \leq t1 \\ P_{Y2}, t1 < t \leq t2 \\ P_{Y3}, t2 < t \end{cases}$$

取圆柱体表面离散点集中的一点 Q

在时刻 t 下，定义：

导弹位置： $M = (M_x, M_y, M_z)$

烟幕中心位置： $C = (C_x, C_y, C_z)$

目标采样位置： $Q = (Q_x, Q_y, Q_z)$

则根据 5.1.2 的几何模型：

$$s = \frac{\overrightarrow{MC} \cdot \overrightarrow{MQ}}{\|\overrightarrow{MQ}\|^2}$$
$$= \frac{(C_x - M_x)(Q_x - M_x) + (C_y - M_y)(Q_y - M_y) + (C_z - M_z)(Q_z - M_z)}{(Q_x - M_x)^2 + (Q_y - M_y)^2 + (Q_z - M_z)^2}$$

确定视线线段上的最近点 H ，由于 $\overrightarrow{MC}$ 的投影应在线段 MQ 上，需要使 $s \in [0,1]$ ，由此得到 s' 。

$$s' = \max(0, \min(1, s))$$

线段 MQ 上距离烟幕中心 C 最近的点 H 的坐标为：

$$H = M + s' \cdot \overrightarrow{MQ}$$

其分量形式为：

$$\begin{cases} H_x = M_x + s' \cdot (Q_x - M_x) \\ H_y = M_y + s' \cdot (Q_y - M_y) \\ H_z = M_z + s' \cdot (Q_z - M_z) \end{cases}$$

计算最短距离 d 并作出最终判定，计算烟幕中心 C 到最近点 H 的欧几里得距离 d

$$d = \sqrt{(C_x - H_x)^2 + (C_y - H_y)^2 + (C_z - H_z)^2}$$

当 $d < 10$ 时，该点被遮蔽。

依照此法求出该时刻所有满足遮蔽条件的点集中的点，计算覆盖率

$coverage(t)$ ：

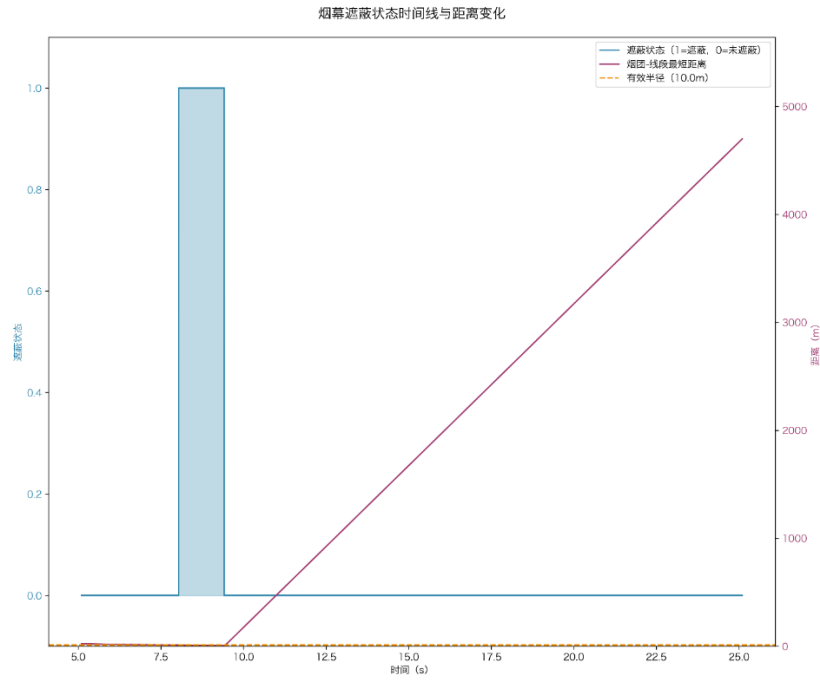
$$coverage(t) = \frac{\text{被遮挡的采样点数量}}{\text{总采样点数量}}$$

阈值 α 取 80%，判断 $coverage(t)$ 是否大于阈值 α ，若大于，则认定此刻圆柱体被遮蔽；否则，未遮蔽圆柱体。

从第一次圆柱体被遮蔽的时刻开始，直到最近一次未被遮蔽的时刻结束，经过的时间即为有效遮蔽时间。将所有的有效遮蔽时间相加，得到总有效遮蔽时长为：

1.409 秒

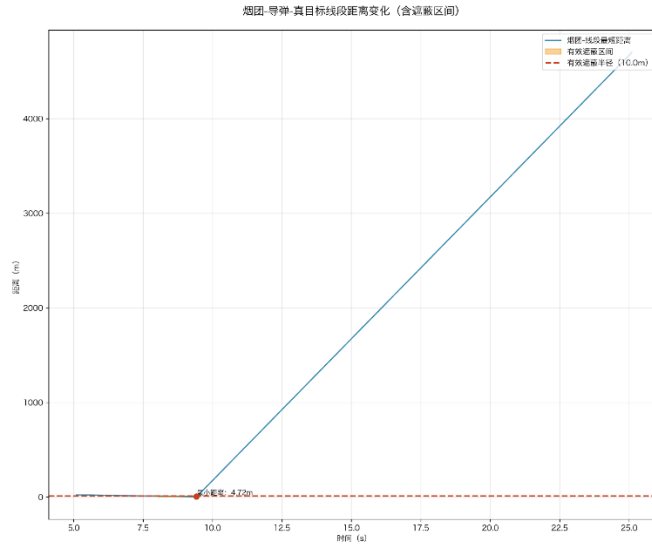
我们还可以根据阈值不同，来计算出不同的总有效遮蔽时长，我们调整遮蔽阈值 50%-99%，计算得出有效遮蔽时长区间为 $[1.392, 1.429]$ ，可以看出因为遮蔽条件的严格，时长明显缩短，后续为了计算便捷，统一采取阈值为 80%，对应的计算结果为 1.407



烟幕遮蔽状态随时间的变化趋势图

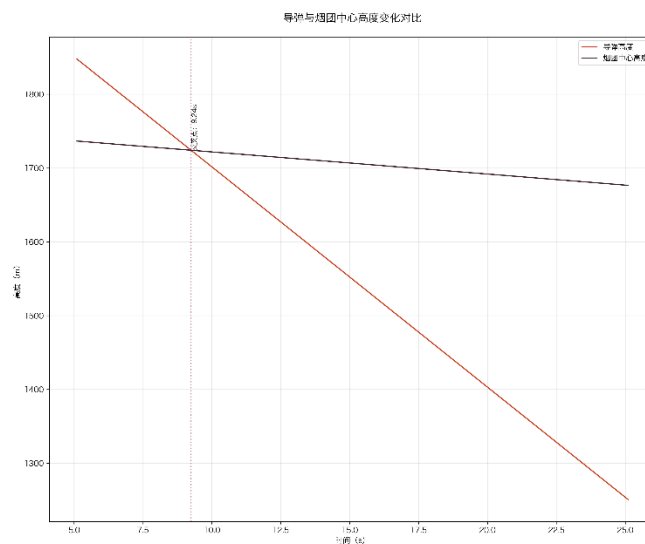
从图中可以看出，烟幕遮蔽状态随时间的变化具有明显的区间特征：

1. 在 $t \approx 8.5 \text{ s}$ 左右距离首次降到 10 m 以内，即遮蔽开始；
2. 附近某一时刻距离最小（接近 0，视线几乎穿过云团中心），遮蔽“最强”；
3. $t \approx 9.5 \text{ s}$ 后距离迅速增大并超过 10 m，即遮蔽结束。
4. 这条距离曲线之后几乎**线性上升**，反映了导弹与云团/目标的相对几何快速偏离，所以有效遮蔽只维持了很短的一段时间。



烟雾弹真目标视线与烟幕云团中心的最小距离随时间的变化曲线

本图展示了导弹—真目标视线与烟幕云团中心的最小距离随时间的变化，并标出遮蔽阈值 10m（红色虚线）及对应的遮蔽区间。蓝色曲线表示距离：在约 8.0 - 9.4s 之间，曲线短暂低于 10m 阈值（图中已标注“遮蔽区间 8.04 - 9.45s”），说明视线被云团有效遮蔽；区间外曲线始终高于阈值，不满足遮蔽判据。此后距离随时间近似线性快速增大，达到数千米，表明导弹与云团在空间上迅速拉开。整体上，图像清晰地给出“仅在约 1.4s 的窗口内存在一次有效遮蔽，其余时段无遮蔽”的结论。



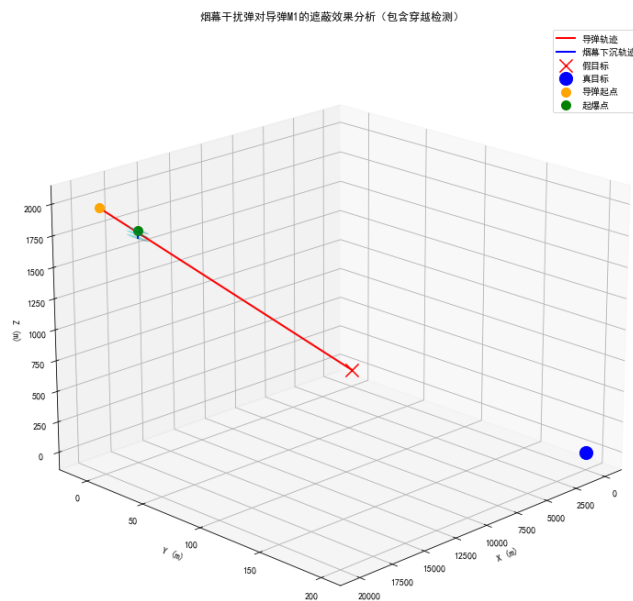
导弹与烟团中心高度变化对比图

本图对比了导弹高度（红线）与烟幕云团中心高度（黑线）随时间的变化。

红线基本是一直线快速下降，表示导弹沿直线飞向地面；它的垂向分量约为 30 m/s 左右，因此 20 s 内高度能降低近 600 m ，下降很快。

黑线缓慢下降，斜率约 -3 m/s ，对应题设“云团中以 3 m/s 匀速下沉”， 20 s 仅下降约 60 m 。图中的竖直虚线标出了两条曲线的交点时刻（ $\approx 8\text{ s}$ 量级）：此刻导弹与云团中心处于同一高度。在此之前导弹在云团上方，之后则在其下方。

“同高”只是满足遮蔽的必要条件之一，遮蔽还取决于水平距离是否足够小（ 3D 距离 $\leq 10\text{ m}$ ）。由于导弹下降快、云团下沉慢，高度仅在交点附近短暂接近；结合水平距离的变化，遮蔽窗口只会出现在交点附近的一小段时间（你前图给出的约 $8.0 - 9.4\text{ s}$ ），之后两者高度迅速拉开，难以再次满足 10 m 判据——因此出现唯一一次、约 1.4 s 的有效遮蔽。



5.2 问题二模型的建立与求解

与问题一相比，问题二中无人机飞行的方向、速度以及烟幕干扰弹的投放地点、起爆时间均未知，不能利用问题一的模型直接正向求解。问题二需要在无数个可行解探索出最优策略，所以必须要设计一种智能搜索算法。

问题一中已经建立了圆柱体遮蔽模型，在此基础上，问题二建立以最长干扰时间为目标函数，以航向角、飞行速度、投放时间（从接到任务开始计时）和引信延时（从投放到起爆的时间间隔）为决策变量的单目标优化模型。由于目标函数涉及复杂的集合判定和时间积分，且决策变量之间存在相互影响，故采用粒子群算法进行优化。

5.2.1 最长干扰时间模型的建立

(1) 运动学方程的建立

由于无人机 FY1 的飞行方向和速度、烟幕干扰弹投放点和烟幕干扰弹起爆点均未知，且都会影响到烟幕弹的最长干扰时间，故以无人机航向角 θ 、飞行速度 v 、投放时刻 t_{drop} 和引信延时 τ 为决策变量。其中：

$$\theta \in [0, 2\pi);$$

$$v \in [70, 140]m/s;$$

$$t_{drop} \in [0, 12]s;$$

$$\tau \in [0, 6]s$$

由以上决策变量可推导出以下变量的表达式：

$$\text{无人机的速度向量 } V = v_{FY1}(\cos\theta, \sin\theta, 0)$$

$$\text{烟幕弹的投放点 } P_{drop} = P_0 + Vt_{drop}$$

$$\text{烟幕弹的起爆点 } C_0 = P_{drop} + V\tau + (0, 0, -0.5 * g * \tau^2)$$

$$\text{起爆时刻 } t_{boom} = t_{drop} + \tau$$

由此可以写出导弹和烟幕中心的各时刻位置：

导弹位置 $M(t) = M(0) + v_{M1} \cdot t$ ，即

$$M(t) = (20000, 0, 2000) + 300 \frac{(-10, 0, -1)}{\sqrt{101}} t$$

$$\text{烟幕中心位置 } C(t) = C_0 + (0, 0, -v_{sink} \cdot \max[0, t - t_{boom}]),$$

(2) 目标函数的建立

建立运动学方程后，按照问题一的求解思路能得到有效遮蔽时长的表达式。

与问题一稍有不同的是，在问题二中设置的极小步长更新到 0.001s，因为第二问是寻找全局最优解，且后续用到的粒子群算法会根据适应度函数调整搜索方向，因此需要误差尽量小。得到目标函数：

$$\max f(\theta, v, t_{drop}, \tau) = \sum_i (t_{end}^{(i)} - t_{start}^{(i)})$$

(3) 约束条件

变量边界约束

航向角： $0 \leq \theta < 2\pi$

飞行速度： $70 \leq v \leq 140m/s$

投放时刻： $0 \leq t_{drop} \leq 12s$

引信延时： $0 \leq \tau \leq 6s$

现实条件约束

$t_{boom} < T_{hit}$ （导弹命中假目标经过的时间）

$t_{boom} \leq t_{valid}$ （烟幕弹有效作用时间）

5.2.2 粒子群算法

(1) 求解过程

在第一问的基础上，我们发现这样的结果不是最优的，针对单烟雾弹的情况，我们需要确定最合理的烟雾弹爆炸时的位置坐标，然后使用粒子群优化算法。

粒子群优化算法（Particle Swarm Optimization，简称 PSO）是一种模拟自然界中鸟群觅食行为的计算方法 PSO 算法是基于群体的，根据对环境的适应度将群体中的个体移动到好的区域，它主要用于解决复杂的优化问题。通过第一问的求解，我们发

现结果不是最优的，于是针对问题二，我们对航向角 θ 、飞行速度 v 、投放时刻 t_{drop} 和引信延时 τ 组成的四维向量 $X_i = [\theta_i, v_i, t_{drop_i}, \tau_i]$ 采用粒子群算法优化，从而进行求解。
算法流程如下：

a. 初始化粒子群及参数

首先定义粒子数目为 64，最大迭代次数为 200 次，惯性权重 $\omega \in [0.4, 0.9]$ 且从最大值起线性递减；个体学习因子为 1.6，群体学习因子为 1.6；在 $[0, 70, 0, 0]$ 与 $[2\pi, 140, 12, 6]$ 之间随机生成位置向量 X 并设置速度边界 $[20, 10, 1.0, 0.6]$

b. 计算粒子适应度

对于粒子 i 的位置向量 $X_i = [\theta_i, v_i, t_{drop_i}, \tau_i]$

粒子 i 的运动方程如下：

$$\overrightarrow{v_{FY}} = v_i [\cos(\theta_i), \sin(\theta_i), 0]$$

$$\overrightarrow{p_{drop}} = \overrightarrow{FY0} + \overrightarrow{v_{FY}} \cdot t_{drop,i}$$

$$t_{boom} = t_{drop_i} + \tau_i$$

$$\overrightarrow{C_0} = \overrightarrow{p_{drop}} + \overrightarrow{v_{FY}} \cdot \tau_i + \frac{1}{2} \vec{g} \tau_i^2$$

时间离散化， $dt=0.001s$ ，可表达为：

$$t_k = k \cdot dt, k = 0, 1, 2, \dots, N$$

其中

$$N = \frac{t_{end}}{dt}, t_{end} = \min(t_{boom} + 20, T_{hit})$$

计算各个时刻的覆盖率 $coverage(t)$ ，通过判断覆盖率是否大于阈值 α 判断该时刻是否遮蔽圆柱体，有效区间提取与时长累加与第一问相同，最终得到遮蔽总时长关于粒子 i 的位置向量的函数，即为适应度函数 $fit(X_i)$

c. 计算个体与全局最优解

先将每个粒子的适应值 $fit(X_i)$ 与其经过最好的适应度 $pbest(X_i)$ 比较，若 $fit(X_i) > pbest(X_i)$ ，则更新 $fit(X_i)$ 为该粒子的个体最优解；再将所有粒子的最

高适应度与全局最高适应度进行比较，若所有粒子的最高适应度大于全局的最高适应度，则将其更新为新的全局最高适应度。

结合个体最优和全局最优，PSO 的完整更新公式为：

速度更新：

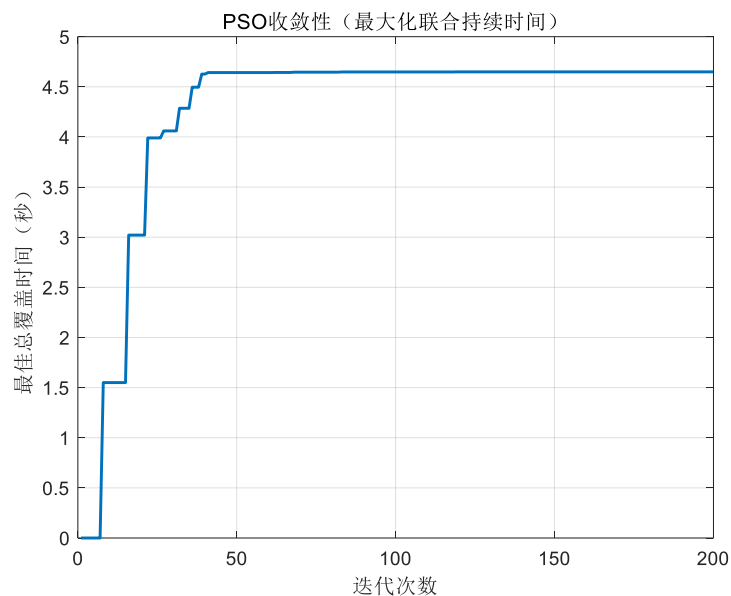
$$v_i^{(t+1)} = \omega \cdot v_i + c_1 \cdot r_1(pbest(i) - X(i)) + c_2 \cdot r_2(gbest(i) - X(i))$$

位置更新：

$$x_i^{(t+1)} = x_i^{(t)} + v_i^{(t+1)}$$

根据上述算法，用粒子群得出的结果为：**4.6 秒**

(2) 结果展示及结果分析



我们画出了 PSO 的收敛曲线，发现相比问题一，最佳总覆盖时间大幅提升。证明了我们的模型计算是真实有效的。优化后的参数实现了烟幕云团和导弹轨迹的最佳交汇。

5.3 问题三模型的建立与求解

5.3.1 问题分析：

问题 3 在问题 2 的基础上更复杂了，问题二只有四个决策变量，分别为无人机的飞行速度，飞行方向角，投弹时间，引爆时间，但是问题三有三枚烟雾弹，引入 8 个决策变量，分别是无人机的飞行速度，飞行方向角，三个投弹时间，三个引爆时间。

同时，我们需要讨论解决一个非常重要的问题：如何计算单个烟雾弹到有效遮蔽时长与三个烟雾弹共同作用下有效遮蔽时间的总时长。经过分析，我们认为单烟雾弹有效遮蔽时长就是该烟雾弹形成的烟雾球幕所在的球心位置与导弹——真目标连线的距离小于球心半径开始计时，同时需要计算这个遮蔽的时间区间，总烟雾弹计时需要取三个烟雾弹遮蔽时间区间的并集，所以会出现有些烟雾弹的有效遮蔽时间为 0，但是它在这一时期的时间区间是有总遮挡时间区间的，或者说就是三个烟雾弹的有效遮蔽时长之和要小于总烟雾弹有效遮蔽时长。

5.3.2 问题建模

针对第三问，我们仍采取粒子群 PSO 方法。

先计算烟雾弹球心的轨迹公式：首先初始化 3 个烟雾弹球心坐标。

FY1 初始点 $P_{FY}(0) = (17800, 0, 1800)$,

速度向量 $V = v[\cos \theta, \sin \theta, 0]$,

速度 $v \in [70, 140]$ m/s

然后计算第 i 枚 ($i=1, 2, 3$) 烟雾弹，投放时刻 $t_{drop\ i}$ ，引爆延时 τ_i ，起爆时刻 $t_{boom\ i} = t_{drop\ i} + \tau_i$

无人机 t 时刻的位置坐标：（等高度直线）

$$P_{FY}(t) = P_{FY}(0) + V t_{drop\ i}$$

第 i 枚弹体（投放后仅受重力，水平速度与无人机一致）：

$$r_i(t) = P_{FY}(0) + V t_{drop\ i} + \left[0, 0, -\frac{1}{2} g \max(t - t_{drop\ i}, 0)^2 \right], \quad t \geq 0.$$

第 i 朵云起爆点：C0i

$$C0i = P0 + vt_{boom,i}d + \left[0,0,-\frac{1}{2}g\tau_i^2\right]$$

第 i 朵云中心轨迹（起爆后 20 s 内有效）：

$$C_i(t) = C_{boom,i} + [0,0,v_z](t - t_{boom,i}), \quad t \in [t_{boom,i}, t_{boom,i} + 20] \text{ s}$$

根据这些解析式，结合 PSO 智能优化算法，就可以计算第三问了

第三问我们需要重点讨论单个烟雾弹的有效遮蔽时长与多个烟雾弹有效遮蔽总时长的关系，我们这里采取的是取每个烟雾弹遮蔽时长的时间区间，对所有的时间区间取并集，得到总有效遮蔽时长，但是在实际操作中，我们发现容易出现这样的问题，即每个烟雾弹的有效遮蔽时长之和小于总有效遮蔽时长，我们认为这个原因是单烟雾弹并没有实现对导弹——目标连线的有效遮蔽，但是两团烟雾区域叠加在一起就可以实现遮蔽，根据这样的原理，我们编写了 Matlab 代码进行计算。最终得出结果大概为 6.1s 左右，无人机的飞行和投弹参数已经按照题目要求保存再 result1 文件中

5.4 问题四模型的建立与求解

第四问是三架无人机各投一枚烟雾弹，最简单的想法是形成了三个烟雾球，而这个烟雾球与导弹与真目标相连的连线的位置关系导致了是否有效遮挡。

问题 4 的主要难点是决策变量太多，每个无人机有四个决策变量，飞行速度，飞行方向，投弹时间，爆炸时间，三个无人机总共 12 个决策变量，为了防止在过高维数上使用 PSO 导致的误差过大和长期陷入局部最优解的问题，我们采取网格搜索法，先缩小搜索范围，之后使用启发式搜索，找到一个更好种子。然后开始 PSO 优化。

5.4.1 问题模型的建立

(1) 决策变量的定义

问题四由问题二的单架无人机转变为三架无人机，按照问题二的建模思路，可将问题二中的四个决策变量扩展成 12 个决策变量：

$$x = [\theta_1, v_1, t_{drop,1}, \tau_1, \theta_2, v_2, t_{drop,2}, \tau_2, \theta_3, v_3, t_{drop,3}, \tau_3]$$

其中下标 1, 2, 3 分别对应无人机 FY1, FY2, FY3。

(2) 目标函数的设计

我们的优化目标是最大化三朵异源烟幕云共同作用下的总有效遮蔽时长。同时，为了使烟幕云的布局更合理，我们引入了“反重叠”惩罚项。目标函数为：

$$\max f(x) = T_{\text{union}}(x) - \lambda \cdot \text{Overlap}(x)$$

其中 $T_{\text{union}}(x)$ 是三朵烟幕云的遮蔽总时长，由三朵云遮蔽时间区间的并集得到； $\text{Overlap}(x)$ 用于衡量三朵云在遮蔽时间上的重合度，计算公式为：

$$\text{Overlap}(x) = \left(\sum_{i=1}^3 T_{\text{indiv},i}(x) \right) - T_{\text{union}}(x)$$

$T_{\text{indiv}, i}(x)$ 是第 i 朵云单独作用时的有效遮蔽时长；

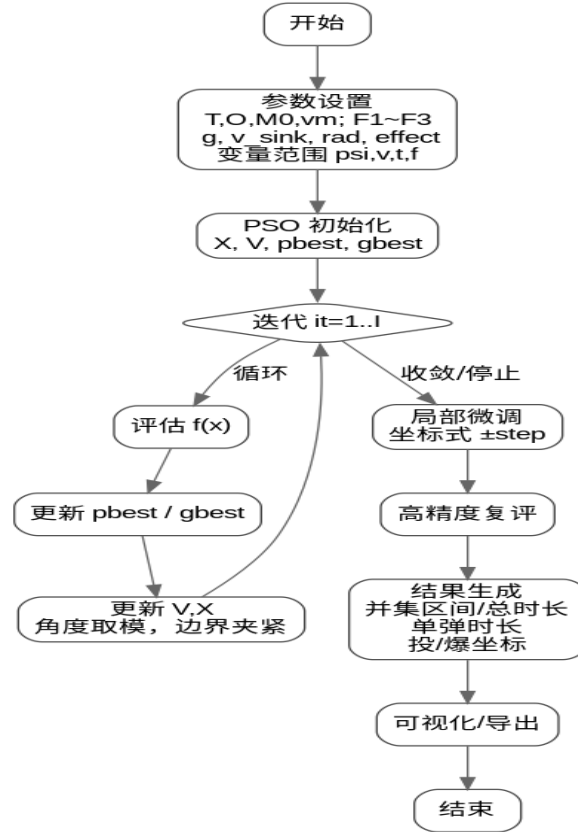
λ 是重叠惩罚系数，用于调节遮蔽总时长 $T_{\text{union}}(x)$ 和时间重合度 $\text{Overlap}(x)$ 的权重。设置 λ 为一个正数，当 $\text{Overlap}(x)$ 越大时使得目标函数越小，越偏离优化方向。最终达到选择遮蔽时间更长、资源利用更高效的策略的目的。

5.4.2 求解策略

由于第四问决策变量更多，直接使用 PSO 算法存在收敛速度慢的问题。为了克服这一难点，采取多阶段优化策略：

下面是一副解决整个第四问的流程图：

总体流程：PSO + 局部微调 + 结果输出



(1) 第一阶段：随机搜索粗筛

由于直接在第四问中采用 PSO 算法存在搜索空间巨大、目标函数复杂等问题，所以在进行 PSO 优化前粗筛选目标。随机选取 5000 个点进行搜索，将 5000 分为 10 组，每组处理 500 个点，在每一组中逐个算出每一点的目标函数值，通过更新可以得到每一组的最优解。若第 i 组的最优解大于全局最优解，则更新全局最优解。将该点作为 PSO 的高质量起点：起点 1。

(2) 第二阶段：多起点 PSO 优化

在第一阶段的基础上，以起点 1 为锚点，通过约束随机生成得到起点 2 到起点 8。每个新起点需要与所有已经生成的起点的距离不小于空间尺度的 20%，确保起点在搜索空间内充分分布。

基于生成的 8 个起点，分别进行 PSO 优化。为了提高优化效果，对 PSO 算法进行以下调整：

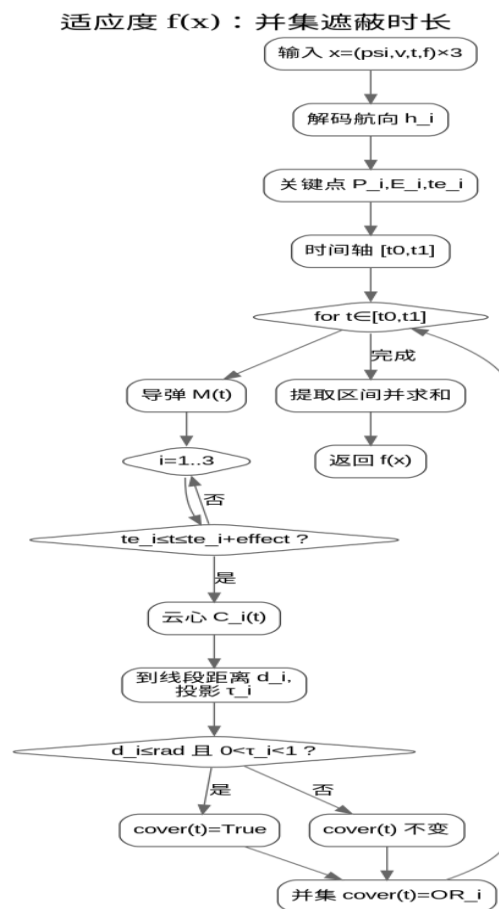
- a. **设置 PSO 自动调整：**当发现连续多轮都没有找到更好的解时，通过调整惯性权

重 ω 、个体和群体学习因子 c_1 c_2 自动扩大搜索范围，尝试跳出局部最优解。

- b. **设置停止条件和阈值：**当某个起点在切换阈值（50 轮）优化内无法达到评分阈值（4.6），该起点的优化停止，优化下一起点；若超过评分阈值，则继续优化至收敛。
- c. **采用分层搜索：**每个 PSO 起点的粒子数量定为 48-96，这些粒子的初始位置采用分层分布，达到同时搜索起点附近、边界和全局的目的。

(3) 第三阶段：局部精调和输出

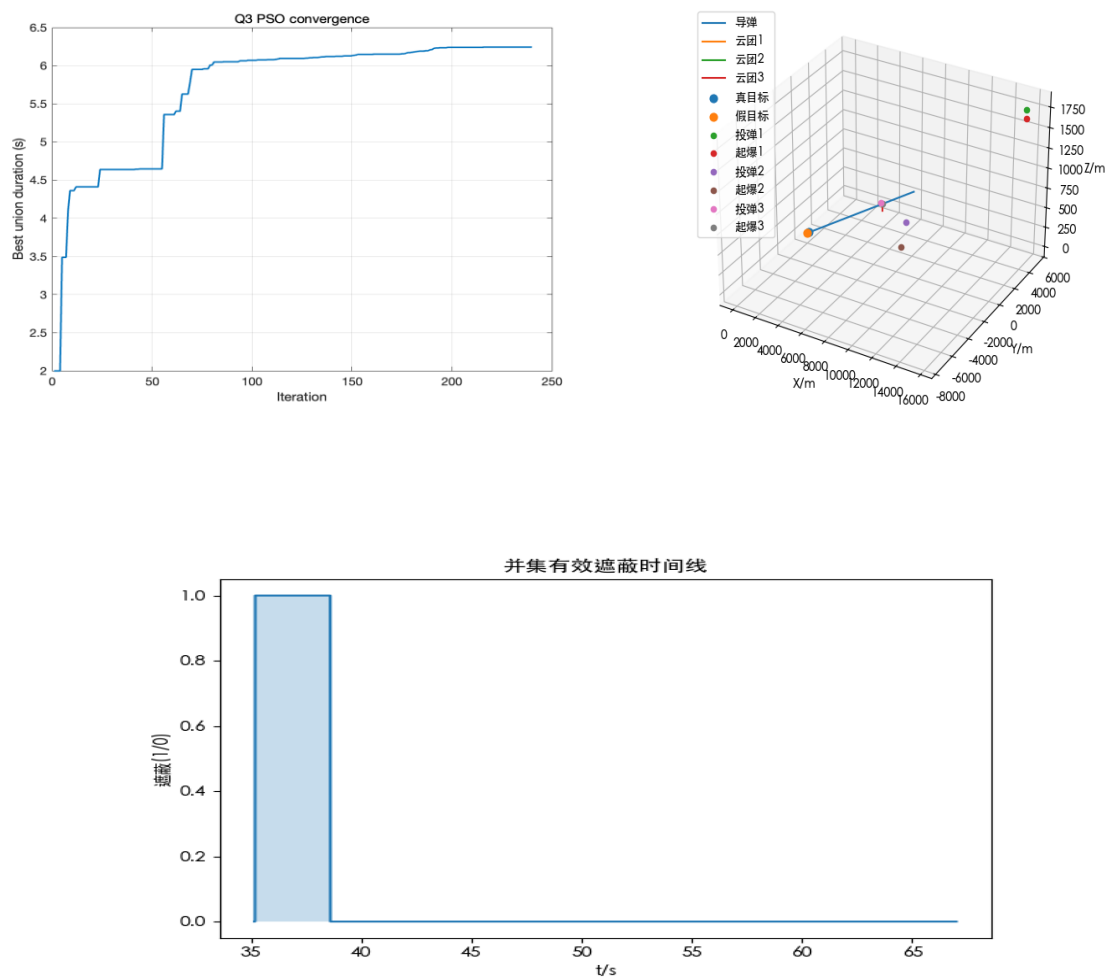
对多起点得到的最终的全局最优解采用坐标下降法进行局部微调，对于每一个决策变量，固定其他变量不变，在该变量的全局最优解附近以极小步长进行搜索（不越过边界），如果得到更好的值就更新这个决策变量的最优解。通过局部精调，可以进一步提升解的质量。最终生成三架无人机的航向角、飞行速度、烟幕弹投放时间和起爆时间投放策略



时长求解流程图

5. 4. 3 问题四的结果输出和分析

对于问题四，我们计算 8.312s 左右，下面是一些结果图：



如图所示，左上图片是我们计算 PSO 过程中的计算图，计算结果大致为 8.1s 左右，右上图片展示的是在遮蔽导弹过程中，我们的实时方位图，下面这幅图展示了我们拦截时长在整个时间轴上的情况。

根据这样的模型设计和计算，我们严谨的解决了问题四, 算得答案为 **8.312s** 左右

5. 5 问题五模型的建立与求解

第五问涉及最多 5 架无人机，每架无人机可投放最多 3 枚烟雾弹，需要针对 3 枚来袭导弹 M1、M2、M3 进行协同遮蔽并求出最长遮蔽时间。问题五的主要难点是变量数目多和复杂的协同决策。每架无人机有 8 个连续决策变量（飞行方向、飞行速度、3 次投

弹时间、3 次爆炸延时)，5 架无人机总共 40 个连续变量。还需要决策每架无人机的 3 枚烟雾弹分别攻击哪个导弹，这涉及离散的目标指派问题。

如果直接应用问题三和问题四的模型会导致优化的搜索空间极大，单纯用 PSO 算法无法解决，计算过程极其复杂。为此，本文提出两阶段分步优化策略，将该问题转化为单机优化和全局指派的组合问题。

5.5.1 问题模型的建立

(1) 决策变量的定义

直接计算将 60 个变量设为决策变量不仅使模型冗杂，而且增加了巨大的计算量。所以在此处分为单机层变量和全局层变量对变量进行分层。

单机层变量为每架无人机 i 的决策变量，由第三问的决策变量可以得到：

$$x_i = [\theta_i, v_i, z_1^i, z_2^i, z_3^i, \tau_1^i, \tau_2^i, \tau_3^i, D_i]$$

其中 $D_i = [d_1^i, d_2^i, d_3^i]$ 为该无人机三枚烟雾弹的分配方式

全局层变量为无人机指派向量，是一个 0-1 矩阵：

$$J = [j_1, j_2, j_3, j_4, j_5], \quad j_i \in \{0, 1\}$$

当 $j_i = 1$ 时，表示无人机 i 被指派

(2) 目标函数的设计

第五问的优化目标函数延用了第四问的思想，将仅对 M1 的遮蔽转变为对 M1、M2 和 M3 的遮蔽优化目标为：

$$\max f(x) = \sum_{q=1}^3 T_{union,q}(x)$$

$q \in \{1, 2, 3\}$ 表示三枚导弹， $T_{union,q}(x)$ 表示所有烟雾弹对导弹 M_q 的遮蔽时间的并集

5.5.2 问题五的求解过程

(1) 单个无人机方案的生成

- a. 枚举出有效的三弹分配方式, 如[1, 1, 2]表示 2 枚烟幕弹干扰 M1, 1 枚烟幕弹干扰 M2
- b. 复用问题三的框架, 将目标函数扩展为多导弹遮蔽时间的并集:

$$f_i(x_i, D_i) = \sum_{q=1}^3 T_{union,q}^i(x_i, D_i)$$

- c. 对每架无人机的每种分配方式进行 PSO 优化和局部精调, PSO 参数选择与问题三相同
- d. 输出候选方案库, 每个方案包括无人机编号、分配方式、最优参数和适应度值。

(2) 全局指派

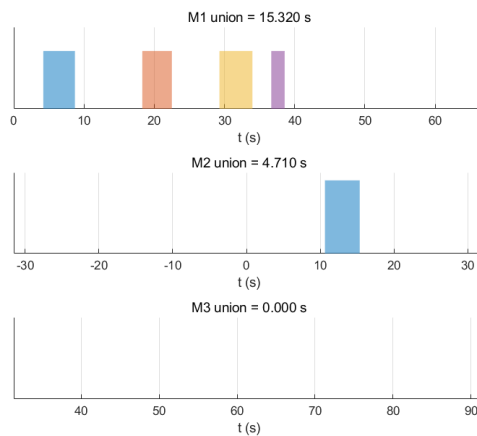
- a. 建立组合选择模型: 在每架无人机的候选方案中至多选择一种, 构成全局作战方案。
- b. 目标函数设计:

$$\max F(S) = \sum_{d=1}^3 T_{union,d}^{global}(S), S \text{ 为方案组合}$$

- c. 在方案包中选择最有价值的组合, 逐步选择剩余方案中与其搭配最好的方案, 以此类推直到选满 5 个方案或选择无明显提高为止。
- d. 局部优化微调, 沿用问题三和问题四的局部微调策略, 对选定方案进行参数精调

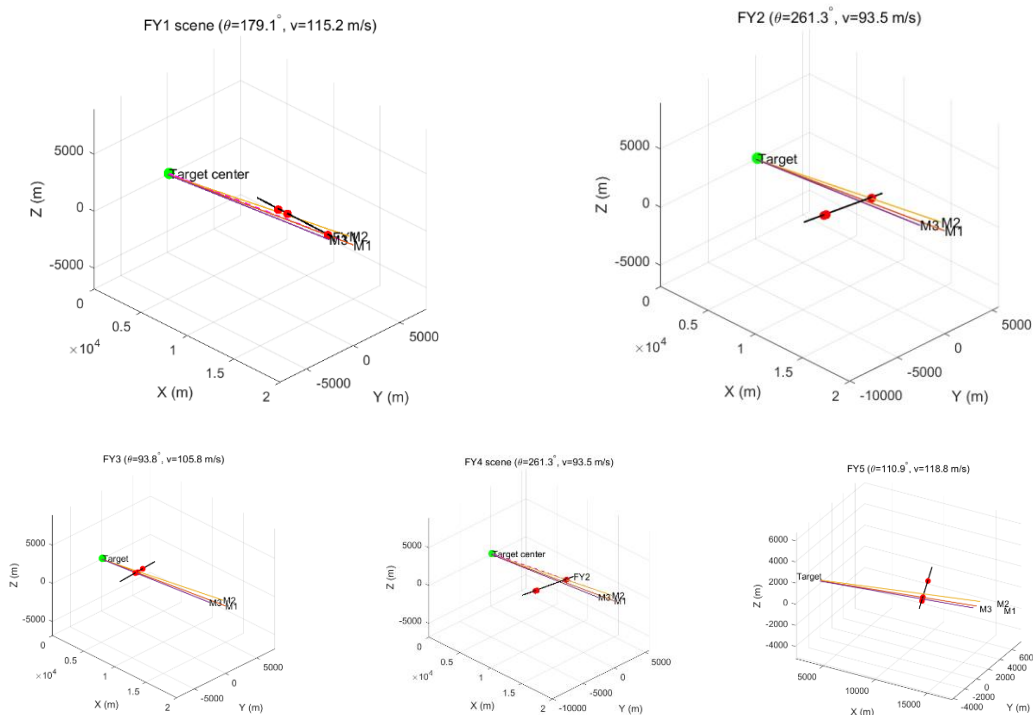
5.5.3 求解结果

通过仿真计算, 求得总共遮挡的时间为 **20.030** 秒 (具体数据已保存至 result3 文件中)



每枚导弹被烟幕遮挡的时间序列

这幅图展示了这三枚导弹被遮挡的情况，从这幅图可以看出 M1 视线被遮挡的效果是比较好的，但是 M3 的视线没有被遮挡。



这几幅图共同展示了这五架无人机的飞行情况和与这三枚导弹的位置关系

六、模型的评价与推广

6.1 模型优点

遮蔽判定机制创新:建立了分层遮蔽判定体系,既提供严格的剪影覆盖率数学标准,又考虑实际工程需求,为烟幕干扰效能评估提供了理论基础。

物理建模理论化:运用空间解析几何理论,将三维烟幕干扰问题转化为严格数学表达,实现现实物理过程的精确理论描述。

智能优化算法优势:广泛采用 PSO 算法具有显著优势:实现简单、收敛快速、初始值依赖性低、全局搜索能力强、适应性好、参数调节灵活,特别适合高维非线性优化。

参数优化改进:对投放时间采用 Sigmoid 映射优化,有效控制参数变化区间,显著提升算法收敛速度和解的稳定性。

分层分解策略:针对超高维问题创新性提出两阶段分解,将多机协同转化为“单机优化+全局调度”,有效控制计算复杂度。

6.2 模型的缺点与不足

高维收敛局限:随着问题复杂度增加,PSO 算法在高维空间易陷入局部最优,搜索空间指数增长使全局搜索能力受限。

算法适应性限制:PSO 算法面对不同规模问题时参数适应性有限,在实践过程中经常出现结果不稳定,参数需要经常调整的情况,需要更具自适应能力的混合优化算法,同时需要设计出可以自动调整参数的优化算法。

附录

附录 1

辅助函数部分：

```
function tf = segment_sphere_hit(A,B,C,R)

% seg-sphere(closest)

a=A; b=B; o=C; r=R;

d=b-a; s=dot(d,d);

if s<1e-12, tf=dot(a-o,a-o)<=r*r; return; end

t=-dot(a-o,d)/s; if t<0, t=0; elseif t>1, t=1; end

q=a+t*d; tf=dot(q-o,q-o)<=r*r;

end
```

SimplePso.m

```
function P = sample_cylinder(nt,nz,caps,r,h,b)

% cyl sample

bx=b(1); by=b(2); bz=b(3);

zs=linspace(0,h,nz);

ts=linspace(0,2*pi,nt+1); ts(end)=[];

P=[];

for z=zs

    for t=ts

        P(end+1,:)= [bx+r*cos(t), by+r*sin(t), bz+z]; %#ok<AGROW>

    end

end

end

if caps
```

```

        nr=max(3,floor(nt/6));

        rs=linspace(0,r,nr);

        st=max(1,floor(nt/12));

        for zc=[bz, bz+h]

            for rr=rs

                for t=ts(1:st:end)

                    P(end+1,:)= [bx+rr*cos(t), by+rr*sin(t), zc]; %ok<AGROW>

                end

            end

        end

    end

end
end

```

q1Main.m

```

% demo

clear; clc;

R=10; k=1-1e-12; dt=0.01; tol=1e-4;

p0=[20000,0,2000]; u=-p0/norm(p0); v=300*u; mpos=@(t) p0+v*t;

cx=0; cy=200; cz=0; r=7; h=10;

xs=[cx-r,cx+r]; ys=[cy-r,cy+r]; zs=[cz,cz+h];

[X,Y,Z]=ndgrid(xs,ys,zs); P=[X(:),Y(:),Z(:)];

cs(1).c_exp=[17188,0,1736.496]; cs(1).t_exp=5.1;

t0=cs(1).t_exp; t1=t0+20;

[iv,tot]=occlusion_intervals(t0,t1,dt,mpos,P,cs,R,k,tol);

```

```

fprintf('--- Box8 ~100%% ---\n');

for i=1:size(iv,1)

    fprintf(' [%8.3f, %8.3f] (%.3f s)\n',iv(i,1),iv(i,2),iv(i,2)-iv(i,1));

end

fprintf('Total: %.3f s\n',tot);

tp=(t0+t1)/2;

[occ, rat, hit, N]=occlusion_ratio_at_time(tp, mpos, P, cs, R, k);

fprintf('t=%.3f -> %d, %.3f (%d/%d)\n', tp, occ, rat, hit, N);

```

Q2Main.m

```

function q2Main()

    % Q2 PSO (single cloud)

    rng(42);

    g=9.8; R=10; k=0.8; dt=0.01; tol=1e-4;

    m0=[20000,0,2000]; uu=-m0/norm(m0); vm=300*uu; mp=@(t) m0+vm*t;

    ms=12;

    P=cylinder_sample_points([0,200,0],7,10,ms);

    lb=[0,70,0.1,0.0]; ub=[2*pi,140,60,6.0];

    o.npop=36; o.iters=240; o.w=0.72; o.c1=1.6; o.c2=1.6; o.vmax=[pi/6,15,2.0,0.8]; o.verbose=true;

    f=@(x) -f_q2(x,mp,P,R,k,dt,tol,g);

    [xb,fb]=simple_pso(f,lb,ub,o);

```

```

th=xb(1); vv=xb(2); te=xb(3); ta=xb(4); tot=-fb;

[iv,tot2]=run_iv(xb,mp,P,R,k,dt,tol,g);

fprintf('\n==== Q2 PSO ==== \n');

fprintf('theta=%0.3f rad (%0.2f deg)\n',th,rad2deg(th));

fprintf('v=%0.3f m/s\n',vv);

fprintf('t_exp=%0.3f s\n',te);

fprintf('tau=%0.3f s\n',ta);

fprintf('total=%0.3f s [chk=%0.3f s]\n',tot,tot2);

fprintf('\n-- intervals --\n');

for i=1:size(iv,1)

    fprintf(' [%0.3f, %0.3f] (%0.3f s)\n',iv(i,1),iv(i,2),iv(i,2)-iv(i,1));

end

end

function T=f_q2(x,mp,P,R,k,dt,tol,g)

th=x(1); v=x(2); te=x(3); ta=x(4);

pen=0;

if ta>te

    pen=1e3*(ta-te);

    ta=min(ta,te-1e-6); if ta<0, ta=0; end

end

p0=[17800,0,1800];

ce=p0+v*te*[cos(th),sin(th),0]+[0,0,-0.5*g*ta^2];

cs(1).c_exp=ce; cs(1).t_exp=te;

t0=te; t1=te+20;

[~,T]=occlusion_intervals(t0,t1,dt,mp,P,cs,R,k,tol);

T=max(T-pen,0);

```



```
end
```

```
function [iv,T]=run_iv(x,mp,P,R,k,dt,tol,g)
```

```
th=x(1); v=x(2); te=x(3); ta=x(4);
```

```
if ta>te, ta=max(0,te-1e-6); end
```

```
p0=[17800,0,1800];
```

```
ce=p0+v*te*[cos(th),sin(th),0]+[0,0,-0.5*g*ta^2];
```

```
cs(1).c_exp=ce; cs(1).t_exp=te;
```

```
t0=te; t1=te+20;
```

```
[iv,T]=occlusion_intervals(t0,t1,dt,mp,P,cs,R,k,tol);
```

```
end
```

```
function [gb,fg]=simple_pso(f,L,U,o)
```

```
d=numel(L); n=o.npop; it=o.iters;
```

```
w=o.w; a=o.c1; b=o.c2; vm=o.vmax; vb=isfield(o,'verbose')&&o.verbose;
```

```
x=L+rand(n,d).*(U-L); v=zeros(n,d); px=x; pf=inf(n,1);
```

```
gb=x(1,:); fg=inf;
```

```
for i=1:n
```

```
    fi=f(x(i,:)); pf(i)=fi; px(i,:)=x(i,:); if fi<fg, fg=fi; gb=x(i,:); end
```

```
end
```

```
if vb, fprintf('init %.6f\n',fg); end
```

```
for k=1:it
```

```
    for i=1:n
```

```
        r1=rand(1,d); r2=rand(1,d);
```

```
        v(i,:)=w*v(i,:)+a*r1.*(px(i,:)-x(i,:))+b*r2.*(gb-x(i,:));
```

```
        v(i,:)=max(min(v(i,:),vm),-vm);
```

```
        xi=x(i,:)+v(i,:); xi=min(max(xi,L),U);
```

```
        fi=f(xi); x(i,:)=xi;
```

```
        if fi<pf(i), pf(i)=fi; px(i,:)=xi; end
```

```

        if fi<fg, fg=fi; gb=xi; end

    end

    if vb&&(k==1||mod(k,10)==0), fprintf('it %d | %.6f\n',k,fg); end

end

end

function d=rad2deg(r), d=r*180/pi; end

```

Q3Main.m

```

function q3Main()

% Q3 PSO (3 drops, cov>=alpha)

g=9.8; R=10; sk=3; eff=20; a=0.80;

o=[0,0,0];

m0=[20000,0,2000]; mv=(o-m0)/norm(o-m0)*300; thit=norm(o-m0)/300;

f0=[17800,0,1800];

nt=24; nz=8; caps=true; dt=0.004; H=25;

np=48; it=240; w1=0.9; w0=0.4; c1=1.6; c2=1.6;

vm=[10,6,1.2,1.2,1.2,0.6,0.6,0.6];

rng(1);

lb=[150,90,-4,-4,-4,0,0,0]; ub=[210,130,4,4,4,6,6,6];

P=sample_cylinder(nt,nz,caps,7,10,[0,200,0]);

D=numel(lb);

X=rand(np,D).*(ub-lb)+lb; V=zeros(np,D);

px=X; pf=-inf(np,1);

gx=zeros(1,D); gf=-inf; hist=zeros(it,1);

```

```

for k=1:it

    w=w1-(w1-w0)*(k-1)/(it-1);

    for i=1:np

        fit=fval(X(i,:),m0,mv,thit,f0,g,R,sk,eff,a,P,dt,H);

        if fit>pf(i), pf(i)=fit; px(i,:)=X(i,:); end

        if fit>gf, gf=fit; gx=X(i,:); end

    end

    r1=rand(np,D); r2=rand(np,D);

    V=w.*V + c1.*r1.*(px-X) + c2.*r2.*(gx-X);

    V=clamp(V,-vm,vm);

    X=clamp(X+V,lb,ub);

    hist(k)=gf;

    if mod(k,20)==0, fprintf('it %d/%d  best=%0.4f s\n',k,it,gf); end

end

[tot,iv,mt]=sim_decode(gx,m0,mv,thit,f0,g,R,sk,eff,a,P,dt,H);

rf=true;

if rf

    mi=@(y) lb+(ub-lb)./(1+exp(-y));

    mo=@(x) log((x-lb)./(ub-x));

    y0=mo(midpoint(gx,lb,ub));

    obj=@(y) -fval(mi(y),m0,mv,thit,f0,g,R,sk,eff,a,P,dt,H);

    op=optimset('Display','off','MaxFunEvals',5000,'MaxIter',3000);

    [yb,fb]=fminsearch(obj,y0,op); xb=mi(yb);

    [t2,iv2,mt2]=sim_decode(xb,m0,mv,thit,f0,g,R,sk,eff,a,P,dt,H);

    if t2>tot, fprintf('refine: %0.3f -> %0.3f s\n',tot,t2); tot=t2; iv=iv2; mt=mt2; gx=xb; gf=t2; end

end

```

```

th=mt.theta; vv=mt.v;

fprintf('\n=== Q3 ===\n');

fprintf('alpha=%.2f | nt=%d nz=%d dt=%.4f | H=%.1f\n',a,nt,nz,dt,H);

fprintf('best=%.3f s | theta=%.2f° | v=%.2f m/s\n',tot,th,vv);

for k=1:3

    fprintf('#%d: t_drop=%.3f tau=%.3f t_exp=%.3f c0=(%.1f,%.1f,%.1f)\n', ...

        k, mt.t_drop(k), mt.tau(k), mt.t_exp(k), mt.c0(k,1), mt.c0(k,2), mt.c0(k,3));

end

disp(iv);

figure('Color','w'); plot(hist,'LineWidth',1.4); grid on;

xlabel('it'); ylabel('best(s)'); title('Q3 PSO');

end

function f=fval(x,m0,mv,thit,f0,g,R,sk,eff,a,P,dt,H)

    [t,~,~]=sim_decode(x,m0,mv,thit,f0,g,R,sk,eff,a,P,dt,H);

    f=t;

end

function [tot,iv,mt]=sim_decode(x,m0,mv,thit,f0,g,R,sk,eff,a,P,dt,H)

    [th,v,t1,u1,t2,u2,t3,u3]=decode(x,H);

    rd=deg2rad(mod(th,360)); fv=[cos(rd),sin(rd),0]*v;

    td=[t1,t2,t3]; uu=[u1,u2,u3];

    te=zeros(1,3); c0=zeros(3,3);

    for k=1:3

        te(k)=td(k)+uu(k);

        pd=f0+fv*td(k);

```

```

        c0(k,:)=pd+fv*uu(k)+[0,0,-0.5*g*uu(k)^2];

    end

    tR=min(max(te+eff),thit); tR=min(tR,H+eff+5);

    ts=0:dt:tR; msk=false(size(ts));

    for k=1:3

        if te(k)>=ts(1) && te(k)<=ts(end)+1e-9

            cov=zeros(size(ts));

            mk=(ts>=te(k))&(ts<=te(k)+eff);

            idx=find(mk);

            for i=idx

                t=ts(i);

                C=c0(k,:)+[0,0,-max(0,t-te(k))*sk];

                M=m0+mv*t;

                d=seg_point_distance_batch(repmat(C,size(P,1),1),repmat(M,size(P,1),1),P);

                cov(i)=mean(d<=R);

            end

            msk = msk | (cov>=a);

        end

    end

    [iv,tot]=mask_to_intervals(msk,ts);

    mt.theta=th; mt.v=v; mt.t_drop=td; mt.tau=uu; mt.t_exp=te; mt.c0=c0;

end

function [th,v,t1,u1,t2,u2,t3,u3]=decode(x,H)

    th=x(1); v=x(2); z1=x(3); z2=x(4); z3=x(5); u1=x(6); u2=x(7); u3=x(8);

    s=@(z) 1./(1+exp(-z));

```

```

H1=max(0,H-2.0); t1=H1*s(z1);

H2=max(0,H-1.0-t1); t2=t1+1.0+H2*s(z2);

H3=max(0,H-1.0-t2); t3=t2+1.0+H3*s(z3);

end

function d=seg_point_distance_batch(P,A,B)

    AB=B-A; AP=P-A; dn=sum(AB.^2,2)+1e-12;

    t=sum(AP.*AB,2)./dn; t=min(max(t,0),1);

    Q=A+AB.*t; d=sqrt(sum((P-Q).^2,2));

end

function [iv,tot]=mask_to_intervals(mk,ts)

    iv=[]; tot=0; in=false; s=0; n=numel(ts);

    for i=1:n

        if mk(i)&&~in, in=true; s=ts(i); end

        if in&&(i==n||~mk(i+1))

            e=ts(i); iv=[iv; s,e]; tot=tot+(e-s); in=false;

        end

    end

end

function P=sample_cylinder(nt,nz,caps,r,h,b)

    bx=b(1); by=b(2); bz=b(3);

    zs=linspace(0,h,nz);

    ts=linspace(0,2*pi,nt+1); ts(end)=[];

    P=[];

    for z=zs

        for t=ts

            P(end+1,:)= [bx+r*cos(t), by+r*sin(t), bz+z]; %#ok<AGROW>

```

```

        end

    end

    if caps

        nr=max(3,floor(nt/6)); rs=linspace(0,r,nr); st=max(1,floor(nt/12));

        for zc=[bz,bz+h]

            for rr=rs

                for t=ts(1:st:end)

                    P(end+1,:)= [bx+rr*cos(t), by+rr*sin(t), zc]; %ok<AGROW>

                end

            end

        end

    end

end

end
end

```

```
function Y=clamp(X,lo,hi), Y=min(max(X,lo),hi); end
```

```
function xm=midpoint(x,lb,ub), e=1e-6; xm=max(min(x,ub-e),lb+e); end
```

Q4Main.m

```

function best = q4Main(varargin)

    % Q4 ultimate PSO

    rng(2025);

    fprintf('=== Q4 决定最终成绩的PSO ===\n');

    if nargin>0

        a=varargin;

        if ischar(a{1}) && strcmp(a{1},'clear_cache')

            clear_cache(); best=[]; return;

        end

    end

```

```

end

g=9.8; R=10; k=0.8; ms=40; dt=0.02; tol=1e-3; lam=0.25; tcap=100;

f0=[0,0,0]; r0=[0,200,0];

m0=[20000,0,2000]; um=(f0-m0)/norm(f0-m0); vm=300*um; mp=@(t) m0+vm*t;

P=cyl_pts(r0,7,10,ms);

p1=[17800,0,1800]; p2=[12000,1400,1400]; p3=[6000,-3000,700];

FY=[p1; p2; p3];

th0=zeros(3,1);

for i=1:3, th0(i)=atan2(-FY(i,2),-FY(i,1)); end

sp=60*pi/180;

lb=[th0(1)-sp,70,0,0, th0(2)-sp,70,0,0, th0(3)-sp,70,0,0];

ub=[th0(1)+sp,140,15,20, th0(2)+sp,140,15,20, th0(3)+sp,140,15,20];

f=@(x) score_fun(x,mp,P,R,k,dt,tol,g,tcap,lam,p1,p2,p3);

fprintf('开始终极多起点优化...\n'); tic;

cf='q4_ultimate_cache.mat'; ok=false; G=struct();

if exist(cf,'file')

    try

        fprintf('发现缓存，加载...\n');

        S=load(cf,'grid_results'); G=S.grid_results;

        if isfield(G,'timestamp')

            age=hours(datetime('now')-G.timestamp);

            if age<12, ok=true; fprintf('缓存OK: best=%.6f (%.1f h)\n',G.fbest,age);

```



```

        else, fprintf('缓存过期(%.1f h)\n',age); end

    else

        fprintf('缓存旧格式\n');

    end

catch e

    fprintf('缓存失败: %s\n',e.message);

end

end

if ~ok

    fprintf('大规模随机搜索(Top-K+早停)...\n');

    [xb,fb]=rand_search(f,lb,ub);

    G=struct('xbest',xb,'fbest',fb,'timestamp',datetime('now'));

    save(cf,'grid_results','-struct','G'); fprintf('已缓存至 %s\n',cf);

end

[xbest,fbest]=multi_start(f,lb,ub,G);

tcost=toc;

fprintf('完成: best=%.6f, 用时=%.2fs\n',fbest,tcost);

[Tun,iv,Tin,drops,cexp,texp]=eval_fun(xbest,mp,P,R,k,dt,tol,g,p1,p2,p3);

th1=xbest(1); v1=xbest(2); t1=xbest(3); ta1=xbest(4);

th2=xbest(5); v2=xbest(6); t2=xbest(7); ta2=xbest(8);

th3=xbest(9); v3=xbest(10); t3=xbest(11); ta3=xbest(12);

fprintf('\n==== Q4 最优 ==== \n');

fprintf('总时=%.2fs\n',tcost);

fprintf('FY1: th=%.4f rad (°.2f°), v=%.3f, t=%.3f, tau=%.3f\n',th1,th1*180/pi,v1,t1,ta1);

fprintf('FY2: th=%.4f rad (°.2f°), v=%.3f, t=%.3f, tau=%.3f\n',th2,th2*180/pi,v2,t2,ta2);

```

```

fprintf('FY3: th=%.4f rad (°.2f°), v=%.3f, t=%.3f, tau=%.3f\n',th3,th3*180/pi,v3,t3,ta3);

for i=1:3

    fprintf('\n-- UAV #%-d --\n',i);

    fprintf(' drop=[%.3f,%.3f,%.3f]\n',drops(i,1),drops(i,2),drops(i,3));

    fprintf(' t_exp=%.3f, c_exp=[%.3f,%.3f,%.3f]\n',texp(i),cexp(i,1),cexp(i,2),cexp(i,3));

    fprintf(' indiv=%.3f s\n',Tin(i));

end

fprintf('\n-- UNION --\n');

for i=1:size(iv,1), fprintf(' [%%.3f, %%.3f] (%.3f)\n',iv(i,1),iv(i,2),iv(i,2)-iv(i,1)); end

fprintf('Union=%.3f s\n',Tun);

fprintf('Obj≈%.3f s\n',Tun - lam*(sum(Tin)-Tun));


best=struct('x',xbest,'theta_deg',[th1,th2,th3]*180/pi,'v',[v1,v2,v3], ...

    't_drop',[t1,t2,t3],'tau',[ta1,ta2,ta3],'t_exp',texp,'drop_pos',drops, ...

    'cexp',cexp,'indiv_time',Tin,'union_time',Tun,'intervals',iv,'optimization_time',tcost);


write_xlsx(best);

fprintf('已写出 result2.xlsx\n');

end


function [xb,fb]=multi_start(f,lb,ub,G)

    D=numel(lb);

    nst=20; tmax=1000; tgt=4.6; sw=100; L1=5.0; L2=8.0; L1m=500; L2m=1000; mind=0.15;

    fprintf('终极多起点: %d 个, 每起点≤%d轮\n',nst,tmax);

    gb=-inf; gx=[]; used=[];

    for s=1:nst

        fprintf('\n--- 起点 %d/%d ---\n',s,nst);

        if s==1

            xs=G.xbest;

```

```

else
    for t=1:200
        xs=lb+rand(1,D).*(ub-lb); ok=true;
        for j=1:size(used,1)
            if norm(xs-used(j,:))/norm(ub-lb)<mind, ok=false; break; end
        end
        if ok, break; end
    end
end

used=[used; xs]; %#ok<AGROW>

[xl,fl,tt]=pso_run(f,lb,ub,xs,tmax,tgt,sw,L1,L2,L1m,L2m);

if fl>gb, gb=fl; gx=xl; fprintf('新全局: %.6f (%d轮)\n',gb,tt); end

if gb>tgt+1.0, fprintf('优秀解, 提前结束\n'); break; end

end

xb=gx; fb=gb; fprintf('\n完成: 全局=%.6f\n',fb);
end

function [xb,fb]=rand_search(f,lb,ub)

D=numel(lb); ns=50000; bs=1000; nb=ceil(ns/bs); K=30;

X=[]; F=[]; noimp=0; best=-inf;

for b=1:nb

    cur=min(bs,ns-(b-1)*bs);

    Xb=lb+rand(cur,D).*(ub-lb); Fb=zeros(cur,1);

    for i=1:cur, Fb(i)=f(Xb(i,:)); end

    X=[X; Xb]; F=[F; Fb]; %#ok<AGROW>

    [F,idx]=sort(F,'descend'); X=X(idx,:);

    if size(X,1)>K, X=X(1:K,:); F=F(1:K); end

    if F(1)>best+1e-9, best=F(1); noimp=0; else, noimp=noimp+1; end

    if mod(b,10)==0 || b==nb, fprintf('随机进度 %d/%d | best=%.6f\n',b,nb,F(1)); end

```

```

        if noimp>=15, fprintf('随机早停\n'); break; end

    end

    xb=X(1,:); fb=F(1);

end

function [xb,fb,tt]=pso_run(f,lb,ub,x0,tmax,tgt,sw,L1,L2,L1m,L2m)

    D=numel(lb); N=300; w1=0.9; w0=0.4; c1=1.6; c2=1.6;

    X=lb+rand(N,D).*(ub-lb); X(1,:)=x0;

    nl=round(N*0.6);

    for i=2:nl

        dlt=0.05*randn(1,D).*(ub-lb);

        xi=x0+dlt; X(i,:)=min(max(xi,lb),ub);

    end

    V=zeros(N,D); PX=X; PF=-inf(N,1);

    for i=1:N, PF(i)=f(X(i,:)); end

    [fg,ii]=max(PF); G=X(ii,:);

    f0=f(x0); if f0>fg, fg=f0; G=x0; end

    cur=100; got1=false; got2=false; st=0; last=fg; tt=0;

    for t=1:tmax

        tt=t;

        if ~got1 && fg>=L1, got1=true; cur=L1m; fprintf('破 %.1f, maxit=%d\n',L1,cur); end

        if ~got2 && fg>=L2, got2=true; cur=L2m; fprintf('破 %.1f, maxit=%d\n',L2,cur); end

        if t>cur

            if fg<L1, fprintf('%d轮未到 %.1f, 切换\n',t-1,L1); break;

            elseif fg<L2 && got1, fprintf('%d轮未到 %.1f, 切换\n',t-1,L2); break;

            else, if t>=1000, fprintf('1000轮到顶, 切换\n'); break; end

        end

    end

end

```

```

    if st>20, w=0.9; else, w=w1-(w1-w0)*(t-1)/max(1,(cur-1)); end

    r1=rand(N,D); r2=rand(N,D);

    V=w*V + c1.*r1.*(PX-X) + c2.*r2.*(G-X);

    vm=[pi/2,35,5,3, pi/2,35,5,3, pi/2,35,5,3]*(1+0.2*(cur-t)/max(1,cur));

    V=max(min(V,vm),-vm);

    X=min(max(X+V,lb),ub);

    if mod(t,10)==0 && st>10

        m=max(1,round(N*0.05)); idx=randperm(N,m);

        X(idx,:)=min(max(X(idx,:)+0.02*randn(m,D).*(ub-lb),lb),ub);

    end

    for i=1:N

        val=f(X(i,:));

        if val>PF(i), PF(i)=val; PX(i,:)=X(i,:); end

        if val>fg, fg=val; G=X(i,:); end

    end

    if fg>last+1e-10, st=0; last=fg; else, st=st+1; end

    if mod(t,10)==0 || t==1

        fprintf('it %3d | score=%0.6f | st=%0d | maxit=%0d\n',t,fg,st,cur);

    end

    if fg<tgt && t>=sw, fprintf('%0d轮未到 %0.1f, 切换\n',t,tgt); break; end

    if st>100, fprintf('停滞 %0d 轮, 切换\n',st); break; end

end

xb=G; fb=fg;

end

function s=score_fun(x,mp,P,R,k,dt,tol,g,tcap,lam,p1,p2,p3)

    th1=x(1); v1=x(2); t1=x(3); ta1=x(4);

    th2=x(5); v2=x(6); t2=x(7); ta2=x(8);

    th3=x(9); v3=x(10); t3=x(11); ta3=x(12);

```

```

te=[t1+ta1, t2+ta2, t3+ta3];

pen=0; tv=[ta1,ta2,ta3]; vv=[v1,v2,v3];

if any(tv<0), pen=pen+100*sum(-tv(tv<0)); end

if any(te>tcap), pen=pen+50*sum(max(te-tcap,0)); end

if any(vv<70)|any(vv>140), pen=pen+5*sum(abs(vv-105)); end


[Tu,~,Ti]=union_fun(th1,v1,t1,ta1, th2,v2,t2,ta2, th3,v3,t3,ta3, mp,P,R,k,dt,tol,g,p1,p2,p3);

ov=max(sum(Ti)-Tu,0);

pref=[8,16]; w=0.05;

cov=mean( min(max(pref(2)-max(pref(1),te),0),20)/20 );

s=Tu - lam*ov - pen + w*cov;

if ~isfinite(s) || s<0, s=0; end

end

function [Tu,Iu,Ti,dp,Ce,Te]=union_fun(th1,v1,t1,ta1, th2,v2,t2,ta2, th3,v3,t3,ta3, mp,P,R,k,dt,tol,g,p1,p2,p3)

p0=[p1; p2; p3];

d=[cos(th1),sin(th1),0; cos(th2),sin(th2),0; cos(th3),sin(th3),0];

vv=[v1,v2,v3]; td=[t1,t2,t3]; tv=[ta1,ta2,ta3];

dp=zeros(3,3); Ce=zeros(3,3); Te=zeros(3,1);

for i=1:3

    v0=vv(i)*d(i,:);

    dp(i,:)=p0(i,:)+td(i)*v0;

    Te(i)=td(i)+tv(i);

    Ce(i,:)=dp(i,:)+tv(i)*v0+[0,0,-0.5*g*tv(i)^2];

end

```

```

Ti=zeros(3,1);

for i=1:3

    C.c_exp=Ce(i,:); C.t_exp=Te(i);

    [~,Ti(i)]=occlusion_intervals(Te(i),Te(i)+20,dt,mp,P,C,R,k,tol);

end

Cs=struct('c_exp',num2cell(Ce,2),'t_exp',num2cell(Te));

[Iu,Tu]=occlusion_intervals(min(Te),max(Te)+20,dt,mp,P,Cs,R,k,tol);

end

function [Tu,iv,Ti,dp,ce,te]=eval_fun(x,mp,P,R,k,dt,tol,g,p1,p2,p3)

    [Tu,iv,Ti,dp,ce,te]=union_fun(x(1),x(2),x(3),x(4),          x(5),x(6),x(7),x(8),          x(9),x(10),x(11),x(12),
mp,P,R,k,dt,tol,g,p1,p2,p3);

end

function [iv,tot]=occlusion_intervals(t0,t1,dt,mp,P,Cs,R,k,tol)

    if ~isstruct(Cs), error('clouds需为struct'); end

    if numel(Cs)==1, Cs=Cs(:); end

    function c=cent(i,t)

        te=Cs(i).t_exp;

        if t<te || t>te+20, c=[NaN,NaN,NaN];

        else, c=Cs(i).c_exp+[0,0,-3*(t-te)]; end

    end

    function tf=occ(t)

        m=mp(t); C=[];

        for i=1:numel(Cs)

```

```

        ci=cent(i,t); if ~any(isnan(ci)), C=[C; ci]; end %#ok<AGROW>

    end

    if isempty(C), tf=false; return; end

    blk=0; Np=size(P,1);

    for j=1:Np

        p=P(j,:); hit=false;

        for i=1:size(C,1)

            if dseg(C(i,:),m,p)<=R, hit=true; break; end

        end

        if hit, blk=blk+1; end

    end

    tf=(blk/Np>=k);

end

T=t0:dt:t1; if T(end)<t1, T=[T,t1]; end

B=false(size(T)); for i=1:numel(T), B(i)=occ(T(i)); end

iv=[]; i=1;

while i<numel(T)

    if ~B(i) && B(i+1)

        a=T(i); b=T(i+1);

        while b-a>tol, m=0.5*(a+b); if occ(m), b=m; else, a=m; end, end

        ts=0.5*(a+b);

        kk=i+1; while kk<numel(T) && B(kk), kk=kk+1; end

        if kk>numel(T), a=T(end-1); b=T(end);

        else, a=T(kk-1); b=T(kk); end

        while b-a>tol, m=0.5*(a+b); if occ(m), a=m; else, b=m; end, end

        te=0.5*(a+b);

        iv=[iv; ts,te]; %#ok<AGROW>

```



```

        i=kk;

    else

        i=i+1;

    end

end

if isempty(iv) && B(1)

    kk=2; while kk<=numel(T) && B(kk), kk=kk+1; end

    a=T(1); b=T(2);

    while b-a>tol, m=0.5*(a+b); if occ(m), b=m; else, a=m; end, end

    ts=0.5*(a+b);

    if kk>numel(T), a=T(end-1); b=T(end);

    else, a=T(kk-1); b=T(kk); end

    while b-a>tol, m=0.5*(a+b); if occ(m), a=m; else, b=m; end, end

    te=0.5*(a+b);

    iv=[ts,te];

end

iv=merge_iv(iv,1e-4);

if isempty(iv), tot=0; else, tot=sum(iv(:,2))-iv(:,1)); end

end

function P=cyl_pts(bc,r,h,ms)

    x0=bc(1); y0=bc(2); z0=bc(3); H=h; R=r;

    nr=max(8,round(ms/4));

    th=linspace(0,2*pi,nr+1); th(end)=[];

    top=[x0+R*cos(th(:)), y0+R*sin(th(:)), z0+H*ones(numel(th),1)];

    bot=[x0+R*cos(th(:)), y0+R*sin(th(:)), z0*ones(numel(th),1)];

    nz=4; zl=linspace(z0,z0+H,nz+2); zl=zl(2:end-1);

```

```

side=[];

for zi=z1, side=[side; x0+R*cos(th(:)), y0+R*sin(th(:)), zi*ones(numel(th),1)]; %#ok<AGROW>

end

box=[x0-R,y0-R,z0; x0-R,y0+R,z0; x0+R,y0-R,z0; x0+R,y0+R,z0; ...
      x0-R,y0-R,z0+H; x0-R,y0+R,z0+H; x0+R,y0-R,z0+H; x0+R,y0+R,z0+H];

P=[top; bot; side; box];

if size(P,1)>max(ms,40)

    idx=randperm(size(P,1),max(ms,40)); P=P(idx,:);

end

end

function write_xlsx(best)

    T=table;

    T.UAV=(1:3).';

    T.theta_deg=best.theta_deg(:);

    T.v=best.v(:);

    T.t_drop=best.t_drop(:);

    T.tau=best.tau(:);

    T.t_exp=best.t_exp(:);

    T.drop_x=best.drop_pos(:,1);

    T.drop_y=best.drop_pos(:,2);

    T.drop_z=best.drop_pos(:,3);

    T.cexp_x=best.cexp(:,1);

    T.cexp_y=best.cexp(:,2);

    T.cexp_z=best.cexp(:,3);

    T.indiv_time=best.indiv_time(:);

    T.union_time=[best.union_time; NaN; NaN];

    writetable(T,'result2.xlsx');

end

```

```

function d=dseg(c,a,b)

    ab=b-a; ac=c-a; L2=dot(ab,ab);

    if L2<=0, d=norm(c-a); return; end

    t=max(0,min(1, dot(ac,ab)/L2));

    q=a+t*ab; d=norm(c-q);

end

function O=merge_iv(I,epsm)

    if isempty(I), O=I; return; end

    I=sortrows(I,1); O=I(1,:);

    for k=2:size(I,1)

        last=O(end,:); cur=I(k,:);

        if cur(1)<=last(2)+epsm, O(end,2)=max(last(2),cur(2));

        else, O=[O; cur]; end %%#ok<AGROW>

    end

end

function clear_cache()

    cf='q4_ultimate_cache.mat';

    if exist(cf,'file'), delete(cf); fprintf('已删 %s\n',cf);

    else, fprintf('无缓存\n'); end

end

```

q4Simple

```

function best = q4Simple()

    % Q4 - simple PSO (serial)

    rng(2025);

    fprintf('=== Q4 简化PSO ===\n');

```

```

if nargin>0
    a=varargin;
    if ischar(a{1}) && strcmp(a{1},'clear_cache'), clear_cache(); return; end
end

g=9.8; R=10; k=0.8; ms=8; dt=0.02; tol=1e-2; lam=0.2; tcap=60;

f0=[0,0,0]; r0=[0,200,0];
m0=[20000,0,2000]; um=(f0-m0)/norm(f0-m0); vm=300*um; mp=@(t) m0+vm*t;

p1=[17800,0,1800]; p2=[12000,1400,1400]; p3=[6000,-3000,700];

P=cylinder_sample_points(r0,7,10,ms);

lb=[0,70,0.0,0.0, 0,70,0.0,0.0, 0,70,0.0,0.0];
ub=[2*pi,140,15.0,8.0, 2*pi,140,15.0,8.0, 2*pi,140,15.0,8.0];

obj=@(x) q4_score_simple(x,mp,P,R,k,dt,tol,g,tcap,lam,p1,p2,p3);

fprintf('开始多起点优化...\n'); tic;
cf='q4_grid_cache.mat'; ok=false; S=[];
if exist(cf,'file')
    try
        fprintf('发现缓存，加载...\n'); load(cf,'grid_results'); S=grid_results;
        if isfield(S,'timestamp')
            age=hours(datetime('now')-S.timestamp);
            if age<24, ok=true; fprintf('缓存OK: best=%.6f (%.1f h)\n',S.fbest,age); else, fprintf('缓存过期(%.1f
h)\n',age); end
        else
            fprintf('缓存旧格式\n');
        end
    catch e
        fprintf('缓存读取失败: %s\n',e.message);
    end
end

if ~ok
    fprintf('开始随机粗筛...\n');
    [xb,fb]=random_search_optimize(obj,lb,ub,mp,P,R,k,dt,tol,g,p1,p2,p3);
    S=struct('xbest',xb,'fbest',fb,'timestamp',datetime('now'));
    save(cf,'grid_results','-struct','S'); fprintf('缓存已写入 %s\n',cf);
end

```

```

[xbest,fbest]=multi_start_optimize(obj,lb,ub,mp,P,R,k,dt,tol,g,p1,p2,p3,S);
tcost=toc;
fprintf('完成: best=%.6f, 用时=%.2fs\n',fbest,tcost);

[Tun, iv, Tin, drops, cexp, texp]=q4_eval_x_simple(xbest,mp,P,R,k,dt,tol,g,p1,p2,p3);

th1=xbest(1); v1=xbest(2); t1=xbest(3); ta1=xbest(4);
th2=xbest(5); v2=xbest(6); t2=xbest(7); ta2=xbest(8);
th3=xbest(9); v3=xbest(10); t3=xbest(11); ta3=xbest(12);

fprintf('\n==== Q4 最优解 ==== \n');
fprintf('总时=%.2fs\n',tcost);
fprintf('FY1: th=%.4f rad (%.2f°), v=%.3f, t=%.3f, tau=%.3f\n',th1,th1*180/pi,v1,t1,ta1);
fprintf('FY2: th=%.4f rad (%.2f°), v=%.3f, t=%.3f, tau=%.3f\n',th2,th2*180/pi,v2,t2,ta2);
fprintf('FY3: th=%.4f rad (%.2f°), v=%.3f, t=%.3f, tau=%.3f\n',th3,th3*180/pi,v3,t3,ta3);

for i=1:3
    fprintf('\n-- UAV # %d --\n',i);
    fprintf(' drop=[%.3f,%.3f,%.3f]\n',drops(i,1),drops(i,2),drops(i,3));
    fprintf(' t_exp=%.3f, c_exp=[%.3f,%.3f,%.3f]\n',texp(i),cexp(i,1),cexp(i,2),cexp(i,3));
    fprintf(' indiv=%.3f s\n',Tin(i));
end
fprintf('\n-- UNION --\n');
for k2=1:size(iv,1), fprintf(' [%.3f, %.3f] (%.3f)\n',iv(k2,1),iv(k2,2),iv(k2,2)-iv(k2,1)); end
fprintf('Union=%.3f s\n',Tun);
fprintf('Obj≈%.3f s\n', Tun - lam*(sum(Tin)-Tun));

best=struct('x',xbest,'theta_deg',[th1,th2,th3]*180/pi,'v',[v1,v2,v3], ...
't_drop',[t1,t2,t3],'tau',[ta1,ta2,ta3],'t_exp',texp,'drop_pos',drops, ...
'cexp',cexp,'indiv_time',Tin,'union_time',Tun,'intervals',iv,'optimization_time',tcost);
end

function [xb,fb]=multi_start_optimize(obj,lb,ub,mp,P,R,k,dt,tol,g,p1,p2,p3,S)
D=numel(lb); nst=8; tmax=500; tgt=5; sw=50; stg=50;
fprintf('多起点: %d 个, 每起点≤%d轮\n',nst,tmax);
gb=-inf; gx=[]; used=[]; mind=0.2;
for s=1:nst
    fprintf('\n--- 起点 %d/%d ---\n',s,nst);
    if s==1
        xs=S.xbest; fs=S.fbest; fprintf('用缓存/粗筛起点: %.6f\n',fs);
    else
        for tries=1:100

```

```

        xs=lb+rand(1,D).*(ub-lb);
        ok=true;
        for j=1:size(used,1)
            if norm(xs-used(j,:))/norm(ub-lb)<mind, ok=false; break; end
        end
        if ok, break; end
    end
    fs=obj(xs);
end
used=[used; xs];
[xl,fl,tt]=simple_pso_optimize_with_limit(obj,lb,ub,mp,P,R,k,dt,tol,g,p1,p2,p3,xs,tmax,tgt,sw,stg);
if fl>gb, gb=fl; gx=xl; fprintf('新全局: %.6f (%d轮)\n',gb,tt); end
if fl<tgt && tt>=sw
    fprintf('%d轮未过 %.1f, 切换\n',tt,tgt);
elseif fl>=tgt
    fprintf('已过 %.1f, 继续至最多 %d 轮\n',tgt,tt);
end
if gb>tgt+0.5, fprintf('优秀解已得, 提前结束\n'); break; end
end
xb=gx; fb=gb; fprintf('\n多起点完成: 全局=%.6f\n',fb);
end

function [xb,fb]=random_search_optimize(obj,lb,ub,mp,P,R,k,dt,tol,g,p1,p2,p3)
    D=numel(lb); ns=5000; bs=500; nb=ceil(ns/bs);
    fprintf('随机搜索 %d 点\n',ns);
    fb=-inf; xb=[];
    for b=1:nb
        cur=min(bs,ns-(b-1)*bs); X=lb+rand(cur,D).*(ub-lb);
        for i=1:cur
            f=obj(X(i,:));
            if f>fb, fb=f; xb=X(i,:); end
        end
        if mod(b,5)==0 || b==nb
            [Tu,~,~]=q4_union_time_simple(xb(1),xb(2),xb(3),xb(4), ...
                xb(5),xb(6),xb(7),xb(8), xb(9),xb(10),xb(11),xb(12), ...
                mp,P,R,k,dt,tol,g,p1,p2,p3);
            fprintf('批 %d/%d | union=%.3f | best=%.6f\n',b,nb,Tu,fb);
        end
    end
    [Tu,~,~]=q4_union_time_simple(xb(1),xb(2),xb(3),xb(4), ...
        xb(5),xb(6),xb(7),xb(8), xb(9),xb(10),xb(11),xb(12), ...
        mp,P,R,k,dt,tol,g,p1,p2,p3);
    fprintf('随机完成: union=%.3f | best=%.6f\n',Tu,fb);
end

```

```

end

function [xb,fb,tt]=simple_pso_optimize_with_limit(obj,lb,ub,mp,P,R,k,dt,tol,g,p1,p2,p3,x0,T,tgt,sw,stg)

    D=numel(lb); N=100; w1=1.2; w0=0.3; c1=2.5; c2=2.5;
    X=lb+rand(N,D).*(ub-lb); X(1,:)=x0;
    nl=round(N*0.5);
    for i=2:nl
        dlt=0.1*randn(1,D).*(ub-lb);
        xi=x0+dlt; X(i,:)=min(max(xi,lb),ub);
    end
    V=zeros(N,D); PX=X; PF=-inf(N,1);
    for i=1:N, PF(i)=obj(X(i,:)); end
    [fg,ii]=max(PF); G=X(ii,:);
    f0=obj(x0); if f0>fg, fg=f0; G=x0; end
    st=0; last=fg; tt=0;
    for t=1:T
        tt=t;
        if st>10, w=1.5; c1=3.0; c2=3.0; else, w=w1-(w1-w0)*(t-1)/(T-1); c1=2.5; c2=2.5; end
        r1=rand(N,D); r2=rand(N,D);
        V=w*V + c1*r1.*(PX-X) + c2*r2.*(G-X);
        vm=[pi/3,25,3,2, pi/3,25,3,2, pi/3,25,3,2]*(1+0.5*(T-t)/T);
        V=max(min(V,vm),-vm);
        X=min(max(X+V,lb),ub);
        if mod(t,10)==0 && st>5
            pidx=randperm(N,round(N*0.1));
            for j=pidx
                xx=X(j,:)+0.1*randn(1,D).*(ub-lb);
                X(j,:)=min(max(xx,lb),ub);
            end
        end
        imp=false;
        for i=1:N
            f=obj(X(i,:));
            if f>PF(i), PF(i)=f; PX(i,:)=X(i,:); imp=true; end
            if f>fg, fg=f; G=X(i,:); imp=true; end
        end
        if fg>last+1e-8, st=0; last=fg; else, st=st+1; end
        if mod(t,5)==0 || t==1
            [Tu,~,~]=q4_union_time_simple(G(1),G(2),G(3),G(4), ...
                G(5),G(6),G(7),G(8), G(9),G(10),G(11),G(12), ...
                mp,P,R,k,dt,tol,g,p1,p2,p3);
            fprintf('Iter %2d | union=%0.3f | best=%0.6f | st=%0d\n',t,Tu,fg,st);
        end
    end
end

```

```

        if fg<tgt && t>=sw, fprintf('%d轮未过 %.1f, 切换\n',t,tgt); break; end
        if fg>=tgt
            if st>500, fprintf('过 %.1f 且停滞 %d 轮, 切换\n',tgt,st); break; end
        else
            if st>stg, fprintf('停滞 %d 轮, 切换\n',st); break; end
        end
    end
end
xb=G; fb=fg;
end

function [xb,fb]=simple_pso_optimize(obj,lb,ub,mp,P,R,k,dt,tol,g,p1,p2,p3,x0)
    D=numel(lb); N=200; T=300; w1=1.2; w0=0.3; c1=2.5; c2=2.5;
    X=lb+rand(N,D).*(ub-lb); X(1,:)=x0;
    nl=round(N*0.4);
    for i=2:nl
        xi=x0+0.1*randn(1,D).*(ub-lb);
        X(i,:)=min(max(xi,lb),ub);
    end
    nb=round(N*0.3);
    for i=(nl+1):(nl+nb)
        if mod(i,2), X(i,:)=lb+0.1*rand(1,D).*(ub-lb); else, X(i,:)=ub-0.1*rand(1,D).*(ub-lb); end
    end
    V=zeros(N,D); PX=X; PF=-inf(N,1);
    for i=1:N, PF(i)=obj(X(i,:)); end
    [fg,ii]=max(PF); G=X(ii,:); f0=obj(x0); if f0>fg, fg=f0; G=x0; fprintf('用粗筛初解\n'); end
    [Tu,~,~]=q4_union_time_simple(G(1),G(2),G(3),G(4), G(5),G(6),G(7),G(8), G(9),G(10),G(11),G(12),
mp,P,R,k,dt,tol,g,p1,p2,p3);
    fprintf('PSO init: union=%.3f | best=%.6f\n',Tu,fg);
    st=0;
    for t=1:T
        if st>10, w=1.5; c1=3.0; c2=3.0; else, w=w1-(w1-w0)*(t-1)/(T-1); c1=2.5; c2=2.5; end
        r1=rand(N,D); r2=rand(N,D);
        V=w*V + c1*r1.*(PX-X) + c2*r2.*(G-X);
        vm=[pi/3,25,3,2, pi/3,25,3,2, pi/3,25,3,2]*(1+0.5*(T-t)/T);
        V=max(min(V,vm),-vm);
        X=min(max(X+V,lb),ub);
        if mod(t,20)==0 && st>5
            pidx=randperm(N,round(N*0.1));
            for j=pidx
                xx=X(j,:)+0.1*randn(1,D).*(ub-lb);
                X(j,:)=min(max(xx,lb),ub);
            end
        end
    end
end

```



```

    imp=false;
    for i=1:N
        f=obj(X(i,:));
        if f>PF(i), PF(i)=f; PX(i,:)=X(i,:); imp=true; end
        if f>fg, fg=f; G=X(i,:); imp=true; end
    end
    if imp, st=0; else, st=st+1; end
    if mod(t,2)==0 || t==1
        [Tu,~,~]=q4_union_time_simple(G(1),G(2),G(3),G(4), G(5),G(6),G(7),G(8), G(9),G(10),G(11),G(12),
mp,P,R,k,dt,tol,g,p1,p2,p3);
        fprintf('it %3d | union=%0.3f | best=%0.6f | st=%0d\n',t,Tu,fg,st);
    end
    if st>30 && fg>0.1, fprintf('早停: st=%0d best=%0.6f\n',st,fg); break; end
end
xb=G; fb=fg;
fprintf('开始局部搜索...\n');
[xb,fb]=local_search_optimize(xb,fb,obj,lb,ub,mp,P,R,k,dt,tol,g,p1,p2,p3);
end

function [xb,fb]=local_search_optimize(xb,fb,obj,lb,ub,mp,P,R,k,dt,tol,g,p1,p2,p3)
    D=numel(lb); T=50; step=0.1; eps=1e-6;
    for it=1:T
        imp=false; base=fb;
        for d=1:D
            x=xb; x(d)=min(max(xb(d)+step*(ub(d)-lb(d)),lb(d)),ub(d));
            f=obj(x); if f>fb, xb=x; fb=f; imp=true; end
            x=xb; x(d)=min(max(xb(d)-step*(ub(d)-lb(d)),lb(d)),ub(d));
            f=obj(x); if f>fb, xb=x; fb=f; imp=true; end
        end
        step=step*(imp*0.8 + ~imp*0.5);
        if step<1e-6 || fb-base<eps, break; end
        if mod(it,10)==0
            [Tu,~,~]=q4_union_time_simple(xb(1),xb(2),xb(3),xb(4), xb(5),xb(6),xb(7),xb(8),
xb(9),xb(10),xb(11),xb(12), mp,P,R,k,dt,tol,g,p1,p2,p3);
            fprintf('loc %2d | union=%0.3f | best=%0.6f\n',it,Tu,fb);
        end
    end
end

function s=q4_score_simple(x,mp,P,R,k,dt,tol,g,tcap,lam,p1,p2,p3)
    th1=x(1); v1=x(2); t1=x(3); ta1=x(4);
    th2=x(5); v2=x(6); t2=x(7); ta2=x(8);
    th3=x(9); v3=x(10); t3=x(11); ta3=x(12);

```

```

te=[t1+ta1, t2+ta2, t3+ta3];
pen=0;
tv=[ta1,ta2,ta3];
vv=[v1,v2,v3];
if any(tv<0), pen=pen+50*sum(-tv(tv<0)); end
if any(te>tcap), pen=pen+20*sum(max(te-tcap,0)); end
if any(vv<70)|any(vv>140), pen=pen+2*sum(abs(vv-105)); end
[Tu,~,Ti]=q4_union_time_simple(th1,v1,t1,ta1, th2,v2,t2,ta2, th3,v3,t3,ta3, mp,P,R,k,dt,tol,g,p1,p2,p3);
ov=max(sum(Ti)-Tu,0);
s=Tu - lam*ov - pen;
if ~isfinite(s) || s<0, s=0; end
end

function [Tu,Iu,Ti,dp,Ce,Te]=q4_union_time_simple(th1,v1,t1,ta1, th2,v2,t2,ta2, th3,v3,t3,ta3,
mp,P,R,k,dt,tol,g,p1,p2,p3)
p0=[p1; p2; p3];
d=[cos(th1),sin(th1),0; cos(th2),sin(th2),0; cos(th3),sin(th3),0];
vv=[v1,v2,v3]; td=[t1,t2,t3]; tv=[ta1,ta2,ta3];
C=struct('c_exp',{},{},{'t_exp',{}});
dp=zeros(3,3); Ce=zeros(3,3); Te=zeros(3,1);
for i=1:3
    dp(i,:)=p0(i,:)+td(i)*vv(i)*d(i,:);
    Te(i)=td(i)+tv(i);
    Ce(i,:)=p0(i,:)+Te(i)*vv(i)*d(i,:)+[0,0,-3*tv(i)];
    C(i).c_exp=Ce(i,:); C(i).t_exp=Te(i);
end
Ti=zeros(3,1);
for i=1:3
    [~,Ti(i)]=occlusion_intervals(Te(i),Te(i)+20,dt,mp,P,C(i),R,k,tol);
end
[Iu,Tu]=occlusion_intervals(min(Te),max(Te)+20,dt,mp,P,C,R,k,tol);
dp=dp; Ce=Ce; Te=Te;
end

function [Tu,iv,Ti,dp,ce,te]=q4_eval_x_simple(x,mp,P,R,k,dt,tol,g,p1,p2,p3)
[Tu,iv,Ti,dp,ce,te]=q4_union_time_simple(x(1),x(2),x(3),x(4), x(5),x(6),x(7),x(8), x(9),x(10),x(11),x(12),
mp,P,R,k,dt,tol,g,p1,p2,p3);
end

function clear_cache()
cf='q4_grid_cache.mat';
if exist(cf,'file'), delete(cf); fprintf('已删 %s\n',cf);
else, fprintf('无缓存\n'); end

```

end

Q5Main.m

```
function q5Main(mode, ext)
% q5 v9 (几何遮蔽+外部解注入)
if nargin<1, mode='q5'; end
if nargin<2, ext=[]; end

g=9.8; R=10.0; sk=3.0; Tef=20.0;
O=[0,0,0];
M0=[20000,0,2000; 19000,600,2100; 18000,-600,1900];
MV=zeros(3,3); Th=zeros(3,1);
for m=1:3
    uh=(O-M0(m,:))/norm(O-M0(m,:));
    MV(m,:)=300*uh;
    Th(m)=norm(O-M0(m,:))/300;
end
U=[17800,0,1800; 12000,1400,1400; 6000,-3000,700; 11000,2000,1800; 13000,-2000,1300];
nU=size(U,1);
P0=[0,200,5];

H=55.0;
pso.pop=48; pso.iters=240; pso.w_max=0.9; pso.w_min=0.4; pso.c1=1.6; pso.c2=1.6;
pso.vmax=[15,10,1.5,1.5,1.5,0.8,0.8,0.8];
pso.lb=[0,70,-4,-4,-4,0,0,0]; pso.ub=[360,140,4,4,4,6,6,6];
dtl=0.004; dtg=0.01;

PAT={ [1 1 1]; [2 2 2]; [3 3 3]; [1 1 2]; [1 1 3]; [2 2 1]; [2 2 3]; [3 3 1]; [3 3 2]; [1 2 3] };

dop=false; plt=true; K=5; do2=true; rng(1);

TS=cell(3,1);
for m=1:3, TS{m}=0:dtg:min(Th(m),H+Tef+5); end

EXT=normalize_external(ext);
if isempty(EXT), EXT=get_hardcoded_external(); end

switch lower(mode)
case 'q3'
    i=1; pat=[1 1 1];
    [sc,pkg]=pso_uav_package_center(U(i,:),M0,MV,Th,pat,g,R,sk,Tef,P0,H,dtl,pso,TS);
    fprintf('Q3: FY1 [%d %d %d] -> %.3f s\n',pat,sc);
    if plt, viz_scene_and_timeline(U,i,pkg,M0,MV,Th,P0,TS,R,sk,Tef); end
```

```

return;
otherwise
    fprintf('Gen pkgs (center-only, strict geom)... \n');
    for i=1:nU
        if ~reachability_ok(U(i,:),M0,MV,H,140,60)
            fprintf(' FY%d weak -> H+=10s \n',i);
            H=H+10;
            for m=1:3, TS {m}=0:dtg:min(Th(m),H+Tef+5); end
        end
    end

    PK=cell(nU,1);
    if dop
        parfor i=1:nU
            A=gen_packages_for_uav(i,U,PAT,M0,MV,Th,g,R,sk,Tef,P0,H,dt1,pso,TS,K);
            B=build_packages_from_external(i,EXT,U,M0,MV,Th,g,R,sk,Tef,P0,H,TS);
            PK {i}=merge_package_lists(A,B);
        end
    else
        for i=1:nU
            A=gen_packages_for_uav(i,U,PAT,M0,MV,Th,g,R,sk,Tef,P0,H,dt1,pso,TS,K);
            B=build_packages_from_external(i,EXT,U,M0,MV,Th,g,R,sk,Tef,P0,H,TS);
            PK {i}=merge_package_lists(A,B);
        end
    end

    fprintf('\nGreedy select... \n');
    [CH, perM, totAll, MASK]=greedy_select_packages(PK,TS);
    if do2
        [CH, perM, totAll, MASK]=two_opt_exchange(CH,PK,TS,MASK,perM,totAll);
    end

    fprintf('\n=== Final === \n');
    for i=1:nU
        p=CH {i};
        if isempty(p)||any(isnan(p.v)), fprintf('FY%d: [idle] \n',i); continue; end
        fprintf('FY%d => [%d %d %d],  $\theta$ =%0.2f°, v=%0.1f \n',i,p.pattern,p.theta,p.v);
        for k=1:3
            fprintf(' #%d: M%d | td=%0.3f, tau=%0.3f, te=%0.3f \n',k,p.m_id(k),p.t_drop(k),p.tau(k),p.t_exp(k));
        end
    end

    fprintf('Union per M(s): M1=%0.3f, M2=%0.3f, M3=%0.3f | TOTAL=%0.3f \n',perM(1),perM(2),perM(3),totAll);

```

```

export_results_excel('result_q5.xlsx',CH,U,M0,MV,P0,TS,R,sk,Tef);

if plt
    for i=1:nU, viz_scene_and_timeline(U,i,CH{i},M0,MV,Th,P0,TS,R,sk,Tef); end
    viz_union_bars(perM,TS,MASK);
end
end
end

function EXT=get_hardcoded_external()
EXT=cell(5,1);
iv=cell(1,3); iv{1}=[2.068 7.257]; iv{2}=[]; iv{3}=[];
EXT{1}(1)=struct('intervals',{iv});
EXT{2}(1)=struct('theta',254.29,'v',115.91,'t_drop',[5.228,24.186,25.186],'tau',[6.852,7.045,5.717],'m_id',[1 1 1]);
EXT{2}(2)=struct('theta',210.15,'v',77.94,'t_drop',[9.198,20.000,21.000],'tau',[16.667,7.444,7.499],'m_id',[2 2 2]);
EXT{3}(1)=struct('theta',106.79,'v',111.32,'t_drop',[23.000,24.995,25.995],'tau',[6.140,10.726,8.109],'m_id',[1 1 1]);
EXT{4}(1)=struct('theta',218.44,'v',87.44,'t_drop',[22.792,24.972,25.972],'tau',[13.872,13.356,13.275],'m_id',[1 1 1]);
EXT{5}(1)=struct('theta',230.34,'v',103.15,'t_drop',[19.352,24.973,25.973],'tau',[5.703,6.945,7.450],'m_id',[1 1 1]);
end

function EXT=normalize_external(ext)
EXT=[];
if isempty(ext), return; end
if isstruct(ext)&&isfield(ext,'by_uav'), EXT=ext.by_uav; else, warning('ext 需含 by_uav'); end
end

function L=build_packages_from_external(i,EXT,U,M0,MV,Th,g,R,sk,Tef,P0,H,TS)
L=[];
if isempty(EXT)||numel(EXT)<i||isempty(EXT{i}), return; end
raw=EXT{i}; tmp=[]; ok=false;
for k=1:numel(raw)
    rk=raw(k);
    if isfield(rk,'intervals')
        pkg=build_pkg_from_intervals(rk.intervals,TS); pkg.uav=i; pkg.pattern=[0 0 0];
    elseif all(isfield(rk,['theta','v','t_drop','tau','m_id']))
        th=rk.theta; v=rk.v; td=reshape(rk.t_drop,1,[]); ta=reshape(rk.tau,1,[]); mid=reshape(rk.m_id,1,[]);
        if numel(td)~=3||numel(ta)~=3||numel(mid)~=3, warning('FY%d 外部条目维度不为3',i); continue; end
        td(1)=max(td(1),0); td(2)=max(td(2),td(1)+1); td(3)=max(td(3),td(2)+1);
        pkg=build_pkg_from_raw(U(i,:),th,v,td,ta,mid,M0,MV,Th,g,R,sk,Tef,P0,H,TS); pkg.uav=i;
    else
        warning('FY%d 外部条目缺字段',i); continue;
    end
    if ~ok, pkg=orderfields(pkg); tmp=pkg; L=pkg; ok=true;
end

```

```

else, pkg=orderfields(pkg,tmp); L(end+1)=pkg; %#ok<AGROW>
end
end
if ~isempty(L), fprintf(' FY%d | +%d ext\n',i,numel(L)); end
end

function pkg=build_pkg_from_intervals(iv,TS)
cov=cell(3,1); sc=0;
for m=1:3
    mk=false(size(TS{m}));
    if ~isempty(iv)&&numel(iv)>=m&&~isempty(iv{m})
        I=iv{m}; if size(I,2)~=2, I=reshape(I,[],2); end
        for r=1:size(I,1)
            s=I(r,1); e=I(r,2);
            mk = mk | (TS{m}>=s & TS{m}<=e);
        end
    end
    cov{m}=mk; [~,lenm]=mask_to_intervals(mk,TS{m}); sc=sc+lenm;
end
pkg=struct('theta',NaN,'v',NaN,'t_drop',[NaN NaN NaN],'tau',[NaN NaN NaN], ...
't_exp',[NaN NaN NaN],'c0',zeros(3,3),'m_id',[0 0 0],'cover_mask',{cov},'score',sc,'pattern',[0 0 0]);
end

function pkg=build_pkg_from_raw(FY0,th,v,td,ta,mid,M0,MV,Th,g,R,sk,Tef,P0,H,TS)
rad=deg2rad(mod(th,360)); vv=[cos(rad),sin(rad),0]*v;
te=td+ta; c0=zeros(3,3);
for s=1:3
    pd=FY0+vv*td(s);
    c0(s,:)=pd+vv*ta(s)+[0,0,-0.5*g*ta(s)^2];
end
cov=cell(3,1); for m=1:3, cov{m}=false(size(TS{m})); end
for s=1:3
    m=mid(s); if m<1, continue; end
    ts=TS{m}; if isempty(ts), continue; end
    mk=false(size(ts)); dtg=ts(2)-ts(1);
    idx=find(ts>=te(s) & ts<=te(s)+Tef);
    for ii=idx
        t=ts(ii); tm=(ii<numel(ts))*(ts(ii)+0.5*dtg);
        C=c0(s,:)+[0,0,-max(0,t-te(s))*sk];
        Cm=c0(s,:)+[0,0,-max(0,tm-te(s))*sk];
        M=M0(m,:)+MV(m,:)*t; Mm=M0(m,:)+MV(m,:)*tm;
        d1=point_to_segment_dist(C,M,P0); d2=point_to_segment_dist(Cm,Mm,P0);
        mk(ii)=min(d1,d2)<=R;
    end
end

```

```

end
cov{m}=cov{m}|mk;
end
per=zeros(3,1); for m=1:3, [~,per(m)]=mask_to_intervals(cov{m},TS{m}); end
pkg=struct('theta',th,'v',v,'t_drop',td,'tau',ta,'t_exp',te,'c0',c0,'m_id',mid,'cover_mask',{cov},'score',sum(per),'pattern',mid);
end

function C=merge_package_lists(a,b)
if isempty(a), C=b; return; end
if isempty(b), C=a; return; end
t=orderfields(a(1));
for k=1:numel(a), a(k)=orderfields(a(k),t); end
for k=1:numel(b), b(k)=orderfields(b(k),t); end
C=[a,b];
end

function L=gen_packages_for_uav(i,U,PAT,M0,MV,Th,g,R,sk,Tef,P0,H,dt,pso,TS,K)
L=[]; ok=false; Tmpl=struct();
for ip=1:numel(PAT)
    pat=PAT{ip};
    [sc,pkg]=pso_uav_package_center(U(i,:),M0,MV,Th,pat,g,R,sk,Tef,P0,H,dt,pso,TS);
    if sc<1e-6
        pkg2=aggressive_refine_package(U(i,:),M0,MV,Th,pat,g,R,sk,Tef,P0,H,dt,pso,TS);
        if pkg2.score>pkg.score, pkg=pkg2; sc=pkg2.score; end
    end
    pkg.score=sc; pkg.pattern=reshape(pat,1,[]); pkg.uav=i;
    if ~ok, pkg=orderfields(pkg); Tmpl=pkg; L=pkg; ok=true;
    else, pkg=orderfields(pkg,Tmpl); L(end+1)=pkg; %#ok<AGROW>
    end
    fprintf(' FY%d | [%d %d %d] -> %.3f s\n',i,pat,sc);
end
if ~isempty(L)
    [~,ord]=sort([L.score],'descend'); L=L(ord(1:min(K,numel(L))));
end
end

function [best,pkg]=pso_uav_package_center(FY0,M0,MV,Th,pat,g,R,sk,Tef,P0,H,dt,pso,TS)
lb=pso.lb; ub=pso.ub; vmax=pso.vmax; n=pso.pop; T=pso.iters; w1=pso.w_max; w0=pso.w_min; c1=pso.c1; c2=pso.c2;
D=numel(lb); X=rand(n,D).*(ub-lb)+lb; V=zeros(n,D); pX=X; pF=-inf(n,1); gX=zeros(1,D); gF=-inf;
for it=1:T
    w=w1-(w1-w0)*(it-1)/(T-1);

```

```

for i=1:n
    f=fitness_center(X(i,:),FY0,M0,MV,Th,pat,g,R,sk,Tef,P0,dt,H,TS);
    if f>pF(i), pF(i)=f; pX(i,:)=X(i,:); end
    if f>gF, gF=f; gX=X(i,:); end
end
r1=rand(n,D); r2=rand(n,D);
V=w.*V + c1.*r1.*(pX-X) + c2.*r2.*(gX-X);
V=clamp(V,-vmax,vmax);
X=clamp(X+V,lb,ub);
end
m_in=@(y) lb+(ub-lb)./(1+exp(-y));
m_out=@(x) log((x-lb)./(ub-x));
y0=m_out(midpoint(gX,lb,ub));
obj=@(y) -fitness_center(m_in(y),FY0,M0,MV,Th,pat,g,R,sk,Tef,P0,dt,H,TS);
opts=optimset('Display','off','MaxFunEvals',5000,'MaxIter',3000);
[yb,fb]=fminsearch(obj,y0,opts); xb=m_in(yb); best=-fb;
[pkg,~]=simulate_multi_center(xb,FY0,M0,MV,Th,pat,g,R,sk,Tef,P0,dt,H,TS);
end

function f=fitness_center(x,FY0,M0,MV,Th,pat,g,R,sk,Tef,P0,dt,H,TS)
[~,per]=simulate_multi_center(x,FY0,M0,MV,Th,pat,g,R,sk,Tef,P0,dt,H,TS);
f=sum(per);
end

function [pkg,per]=simulate_multi_center(x,FY0,M0,MV,Th,pat,g,R,sk,Tef,P0,dt,H,TS)
[th,v,t1,ta1,t2,ta2,t3,ta3]=decode_params(x,H);
rad=deg2rad(mod(th,360)); vv=[cos(rad),sin(rad),0]*v;
td=[t1,t2,t3]; ta=[ta1,ta2,ta3]; mid=pat(:)'; te=zeros(1,3); c0=zeros(3,3);
for k=1:3
    te(k)=td(k)+ta(k);
    pd=FY0+vv*td(k);
    c0(k,:)=pd+vv*ta(k)+[0,0,-0.5*g*ta(k)^2];
end
cov=cell(3,1); for m=1:3, cov{m}=false(size(TS{m})); end
for k=1:3
    m=mid(k); ts=TS{m}; if m<1||isempty(ts), continue; end
    mk=false(size(ts)); dd=ts(2)-ts(1);
    idx=find(ts>=te(k) & ts<=te(k)+Tef);
    for ii=idx
        t=ts(ii); tm=(ii<numel(ts))*(ts(ii)+0.5*dd);
        C=c0(k,:)+[0,0,-max(0,t-te(k))*sk];
        Cm=c0(k,:)+[0,0,-max(0,tm-te(k))*sk];
        M=M0(m,:)+MV(m,:)*t; Mm=M0(m,:)+MV(m,:)*tm;
    end
end

```



```

        d1=point_to_segment_dist(C,M,P0); d2=point_to_segment_dist(Cm,Mm,P0);
        mk(ii)=min(d1,d2)<=R;
    end
    cov{m}=cov{m}|mk;
end
per=zeros(3,1); for m=1:3, [~,per(m)]=mask_to_intervals(cov{m},TS{m}); end
pkg=struct('theta',th,'v',v,'t_drop',td,'tau',ta,'t_exp',te,'c0',c0,'m_id',mid,'cover_mask',{cov},'score',sum(per),'pattern',mid);
end

function [CH,perM,tot,mask]=greedy_select_packages(PK,TS)
nU=numel(PK); mask=cell(3,1); for m=1:3, mask{m}=false(size(TS{m})); end
CH=cell(nU,1); pick=false(nU,1);
while true
    best=-inf; bi=-1; bj=-1;
    for i=find(~pick).
        C=PK{i};
        for j=1:numel(C)
            gain=0;
            for m=1:3
                gain=gain + sum(~mask{m} & C(j).cover_mask{m})*(TS{m}(2)-TS{m}(1));
            end
            if gain>best, best=gain; bi=i; bj=j; end
        end
    end
    if best<=1e-9||bi<0, break; end
    CH{bi}=PK{bi}(bj);
    for m=1:3, mask{m}=mask{m} | CH{bi}.cover_mask{m}; end
    pick(bi)=true; if all(pick), break; end
end
for i=1:nU
    if isempty(CH{i})
        CH{i}=struct('pattern',[0 0 0],'theta',NaN,'v',NaN,'t_drop',[NaN NaN NaN],'tau',[NaN NaN NaN], ...
            't_exp',[NaN NaN NaN],'m_id',[0 0 0],'cover_mask',{false(size(TS{1})),false(size(TS{2})),false(size(TS{3}))},'score',0);
    end
end
perM=zeros(3,1); tot=0;
for m=1:3, [~,perM(m)]=mask_to_intervals(mask{m},TS{m}); tot=tot+perM(m); end
end

function [CH,perM,tot,mask]=two_opt_exchange(CH,PK,TS,mask,perM,tot)
imp=true; pass=0; mx=2;

```

```

while imp && pass<mx
    imp=false; pass=pass+1;
    for i=1:numel(PK)
        for a=1:numel(PK{i})
            c=PK{i}(a); gain=0;
            for m=1:3, gain=gain + sum(~mask{m} & c.cover_mask{m})*(TS{m}(2)-TS{m}(1)); end
            if gain>1e-6
                CH{i}=c; [mask,perM,tot]=rebuild_covered_from_chosen(CH,TS); imp=true; break;
            end
        end
    end
    if imp, break; end
end
end
end

function [mask,perM,tot]=rebuild_covered_from_chosen(CH,TS)
mask=cell(3,1); for m=1:3, mask{m}=false(size(TS{m})); end
for i=1:numel(CH), for m=1:3, mask{m}=mask{m} | CH{i}.cover_mask{m}; end, end
perM=zeros(3,1); tot=0;
for m=1:3, [~,perM(m)]=mask_to_intervals(mask{m},TS{m}); tot=tot+perM(m); end
end

function export_results_excel(fn,CH,U,M0,MV,P0,TS,R,sk,Tef)
rows=[];
for i=1:numel(CH)
    p=CH{i}; if isempty(p)||any(isnan(p.v)), continue; end
    th=p.theta; v=p.v; FY0=U(i,:); rad=deg2rad(th); vv=[cos(rad),sin(rad),0]*v;
    for k=1:3
        m=p.m_id(k); if m<1, continue; end
        td=p.t_drop(k); ta=p.tau(k); te=p.t_exp(k);
        pd=FY0+vv*td; c0=p.c0(k,:);
        len=single_shot_length(FY0,vv,td,ta,m,M0,MV,P0,TS{m},R,sk,Tef);
        rows=[rows; i,th,v,k,pd(1),pd(2),pd(3),c0(1),c0(2),c0(3),len,m];
    end
end
names={'无人机编号','无人机运动方向','无人机运动速度(m/s)','烟幕干扰弹编号', ...
'烟幕干扰弹投放点的x坐标(m)','烟幕干扰弹投放点的y坐标(m)','烟幕干扰弹投放点的z坐标(m)', ...
'烟幕干扰弹起爆点的x坐标(m)','烟幕干扰弹起爆点的y坐标(m)','烟幕干扰弹起爆点的z坐标(m)', ...
'有效干扰时长(s)','干扰的导弹编号'};
if ~isempty(rows)
    T=array2table(rows,'VariableNames',names); writetable(T,fn,'FileType','spreadsheet');
    fprintf('Excel: %s (%d rows)\n',fn,height(T));
else

```

```

    fprintf('Excel: empty\n');
end
end

function len=single_shot_length(FY0,vv,td,ta,m,M0,MV,P0,ts,R,sk,Tef)
te=td+ta; mk=fals

```

绘图代码: p1_problem_2

```

# -*- coding: utf-8 -*-
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import font_manager as fm
from pathlib import Path
from datetime import datetime

def use_cn_font():
    candidates = [
        "PingFang SC", "Hiragino Sans GB", "Heiti SC", "Songti SC", "STSong",
        "Microsoft YaHei", "SimHei", "WenQuanYi Zen Hei",
        "Noto Sans CJK SC", "Source Han Sans SC"
    ]
    have = {f.name for f in fm.fontManager.ttflist}
    for name in candidates:
        if name in have:
            plt.rcParams["font.family"] = "sans-serif"
            plt.rcParams["font.sans-serif"] = [name]
            break
    plt.rcParams["axes.unicode_minus"] = False

use_cn_font()
plt.rcParams['figure.figsize'] = (12, 10)
plt.rcParams['lines.linewidth'] = 1.5

#基础工具函数
def normalize_vector(vec):
    v = np.asarray(vec, dtype=np.float64)
    n = np.linalg.norm(v)
    return v / n if n != 0 else v

def calc_point_to_segment_dist(p, a, b):
    a = np.array(a, dtype=np.float64)
    b = np.array(b, dtype=np.float64)
    p = np.array(p, dtype=np.float64)

```

```

ab = b - a
ap = p - a
ab2 = np.dot(ab, ab)
if ab2 < 1e-8:
    tau = 0.0
    closest = a
else:
    tau = np.dot(ap, ab) / ab2
    tau = np.clip(tau, 0.0, 1.0)
    closest = a + tau * ab
dist = np.linalg.norm(p - closest)
return dist, tau

#运动模型函数
def get_missile_position(t, init_pos, target_pos, speed):
    d = normalize_vector(np.array(target_pos) - np.array(init_pos))
    return np.array(init_pos) + speed * t * d

def get_uav_heading_xy(from_pos, to_pos):
    xy = np.array([to_pos[0] - from_pos[0], to_pos[1] - from_pos[1], 0.0])
    return normalize_vector(xy)

def calc_bomb_release_pos(uav_init_pos, uav_speed, heading_vec, release_delay):
    return np.array(uav_init_pos) + uav_speed * release_delay * np.array(heading_vec)

def calc_explosion_pos(release_pos, uav_speed, heading_vec, fuse_delay, gravity=9.8):
    horiz = uav_speed * fuse_delay * np.array(heading_vec)
    vert = -0.5 * gravity * (fuse_delay ** 2)
    return release_pos + np.array([horiz[0], horiz[1], vert])

def get_smoke_center(t, explode_pos, explode_time, sink_speed=3.0):
    dt = t - explode_time
    return np.array([explode_pos[0], explode_pos[1], explode_pos[2] - sink_speed * dt])

#核心计算流程
def calculate_smoke_coverage(time_step=0.001):
    scene = {
        "decoy_pos": np.array([0.0, 0.0, 0.0]),
        "real_target_pos": np.array([0.0, 200.0, 0.0]),
        "missile_init": np.array([20000.0, 0.0, 2000.0]),
        "missile_speed": 300.0,
        "uav_init": np.array([17800.0, 0.0, 1800.0]),
        "uav_speed": 120.0,

```

```

        "release_delay": 1.5,
        "fuse_delay": 3.6,
        "gravity": 9.8,
        "smoke_sink_speed": 3.0,
        "effective_radius": 10.0,
        "smoke_valid_time": 20.0
    }

    uav_heading = get_uav_heading_xy(scene["uav_init"], scene["decoy_pos"])
    release_pos = calc_bomb_release_pos(scene["uav_init"], scene["uav_speed"], uav_heading,
scene["release_delay"])
    explode_pos = calc_explosion_pos(release_pos, scene["uav_speed"], uav_heading, scene["fuse_delay"],
scene["gravity"])
    explode_time = scene["release_delay"] + scene["fuse_delay"]

    t0, t1 = explode_time, explode_time + scene["smoke_valid_time"]
    ts = np.arange(t0, t1 + 0.5 * time_step, time_step)

    cover = np.zeros_like(ts, dtype=bool)
    dmin = np.zeros_like(ts, dtype=np.float64)
    tau_arr = np.zeros_like(ts, dtype=np.float64)
    traj_m = np.zeros((len(ts), 3), dtype=np.float64)
    traj_s = np.zeros((len(ts), 3), dtype=np.float64)

    for i, t in enumerate(ts):
        pm = get_missile_position(t, scene["missile_init"], scene["decoy_pos"], scene["missile_speed"])
        ps = get_smoke_center(t, explode_pos, explode_time, scene["smoke_sink_speed"])
        dist, tau = calc_point_to_segment_dist(ps, pm, scene["real_target_pos"])
        cover[i] = (dist <= scene["effective_radius"]) and (0.0 < tau < 1.0)
        dmin[i] = dist
        tau_arr[i] = tau
        traj_m[i] = pm
        traj_s[i] = ps

    intervals, on, t_start = [], False, 0.0
    for i in range(len(ts)):
        if cover[i] and not on:
            on, t_start = True, ts[i]
        elif (not cover[i]) and on:
            on = False
            intervals.append((t_start, ts[i - 1]))
    if on:
        intervals.append((t_start, ts[-1]))

```

```

total_time = sum(e - s for s, e in intervals)

return {
    "scene_params": scene,
    "bomb_release_pos": release_pos,
    "explosion_pos": explode_pos,
    "explosion_time": explode_time,
    "time_series": ts,
    "cover_status": cover,
    "min_distance": dmin,
    "project_param": tau_arr,
    "missile_trajectory": traj_m,
    "smoke_trajectory": traj_s,
    "cover_intervals": intervals,
    "total_cover_time": total_time
}

#可视化函数
def plot_cover_timeline(time_series, cover_status, min_distance, effective_radius):
    fig, ax1 = plt.subplots()
    ax1.plot(time_series, cover_status.astype(int), color='#2E86AB', label='遮蔽状态（1=遮蔽，0=未遮蔽）')
    ax1.fill_between(time_series, 0, cover_status.astype(int), color='#2E86AB', alpha=0.3)
    ax1.set_xlabel('时间（s）'); ax1.set_ylabel('遮蔽状态', color='#2E86AB')
    ax1.tick_params(axis='y', labelcolor='#2E86AB'); ax1.set_ylim(-0.1, 1.1)

    ax2 = ax1.twinx()
    ax2.plot(time_series, min_distance, color='#A23B72', label='烟团-线段最短距离')
    ax2.axhline(y=effective_radius, color='#F18F01', linestyle='--', label=f'有效半径（{effective_radius}m）')
    ax2.set_ylabel('距离（m）', color='#A23B72'); ax2.tick_params(axis='y', labelcolor='#A23B72')
    ax2.set_ylim(0, max(min_distance) * 1.2)

    l1, lab1 = ax1.get_legend_handles_labels()
    l2, lab2 = ax2.get_legend_handles_labels()
    ax1.legend(l1 + l2, lab1 + lab2, loc='upper right')
    ax1.set_title('烟幕遮蔽状态时间线与距离变化', fontsize=14, pad=20)
    plt.tight_layout()
    return fig

def plot_xy_trajectory(missile_traj, smoke_traj, decoy_pos, real_target_pos, release_pos, explosion_pos):
    fig, ax = plt.subplots()
    ax.plot(missile_traj[:, 0], missile_traj[:, 1], color='#C73E1D', label='导弹轨迹')
    ax.plot(smoke_traj[:, 0], smoke_traj[:, 1], color='#3B1F2B', label='烟团中心轨迹')
    ax.scatter(decoy_pos[0], decoy_pos[1], s=80, color='#F18F01', label='假目标O', zorder=5)

```

```

ax.scatter(real_target_pos[0], real_target_pos[1], s=80, color='#2E86AB', label='真目标T', zorder=5)
ax.scatter(release_pos[0], release_pos[1], s=60, color='#A23B72', marker='^', label='投弹点', zorder=5)
ax.scatter(explosion_pos[0], explosion_pos[1], s=60, color='#C73E1D', marker='*', label='起爆点', zorder=5)
ax.text(missile_traj[0, 0], missile_traj[0, 1], '导弹起点', fontsize=9, ha='right')
ax.text(smoke_traj[0, 0], smoke_traj[0, 1], '烟团起点', fontsize=9, ha='right')
ax.set_xlabel('X轴坐标 (m)'); ax.set_ylabel('Y轴坐标 (m)')
ax.legend(loc='upper left'); ax.set_title('XY平面轨迹: 导弹、烟团与关键目标', fontsize=14, pad=20)
ax.grid(True, alpha=0.3); plt.tight_layout()
return fig

def plot_distance_curve(time_series, min_distance, cover_status, effective_radius):
    fig, ax = plt.subplots()
    ax.plot(time_series, min_distance, color='#2E86AB', label='烟团-线段最短距离')
    mask = cover_status
    ax.fill_between(time_series[mask], 0, min_distance[mask], color='#F18F01', alpha=0.4, label='有效遮蔽区间')
    ax.axhline(y=effective_radius, color='#C73E1D', linestyle='--', linewidth=2,
label=f'有效遮蔽半径 ({effective_radius}m)')
    i_min = np.argmin(min_distance)
    ax.scatter(time_series[i_min], min_distance[i_min], color='#C73E1D', s=50, zorder=5)
    ax.text(time_series[i_min], min_distance[i_min], f'最小距离: {min_distance[i_min]:.2f}m', fontsize=9, ha='left',
va='bottom')
    ax.set_xlabel('时间 (s)'); ax.set_ylabel('距离 (m)')
    ax.legend(loc='upper right'); ax.set_title('烟团-导弹-真目标线段距离变化 (含遮蔽区间)', fontsize=14, pad=20)
    ax.grid(True, alpha=0.3); plt.tight_layout()
    return fig

def plot_height_comparison(time_series, missile_traj, smoke_traj):
    fig, ax = plt.subplots()
    ax.plot(time_series, missile_traj[:, 2], color='#C73E1D', label='导弹高度')
    ax.plot(time_series, smoke_traj[:, 2], color='#3B1F2B', label='烟团中心高度')
    diff = missile_traj[:, 2] - smoke_traj[:, 2]
    cross_idx = np.where(np.diff(np.sign(diff)))[0]
    for i in cross_idx:
        ax.axvline(x=time_series[i], color='#A23B72', linestyle=':', alpha=0.7)
        ax.text(time_series[i], max(missile_traj[i, 2], smoke_traj[i, 2]), f'交叉点: {time_series[i]:.2f}s',
        fontsize=9, rotation=90, va='bottom')
    ax.set_xlabel('时间 (s)'); ax.set_ylabel('高度 (m)')
    ax.legend(loc='upper right'); ax.set_title('导弹与烟团中心高度变化对比', fontsize=14, pad=20)
    ax.grid(True, alpha=0.3); plt.tight_layout()
    return fig

#主程序入口含 PNG 保存
if __name__ == "__main__":

```

```

res = calculate_smoke_coverage(time_step=0.001)

print("【烟幕遮蔽效果】")
print(f'1) 投弹点: ({res["bomb_release_pos"][0]:.3f}, {res["bomb_release_pos"][1]:.3f}, {res["bomb_release_pos"][2]:.3f}) m')
print(f'2) 起爆点: ({res["explosion_pos"][0]:.3f}, {res["explosion_pos"][1]:.3f}, {res["explosion_pos"][2]:.3f}) m')
print(f'3) 起爆时刻: {res["explosion_time"]:.3f} s')
if res['cover_intervals']:
    print("4) 有效遮蔽时间窗(秒):")
    for i, (s, e) in enumerate(res['cover_intervals'], 1):
        print(f'    # {i}: [{s:.3f}, {e:.3f}], 时长: {e - s:.3f} s')
else:
    print("4) 未检测到有效遮蔽区间")
print(f'5) 总有效遮蔽时长: {res["total_cover_time"]:.3f} s')

# 生成图
fig1 = plot_cover_timeline(
    time_series=res['time_series'],
    cover_status=res['cover_status'],
    min_distance=res['min_distance'],
    effective_radius=res['scene_params']['effective_radius']
)
fig2 = plot_xy_trajectory(
    missile_traj=res['missile_trajectory'],
    smoke_traj=res['smoke_trajectory'],
    decoy_pos=res['scene_params']['decoy_pos'],
    real_target_pos=res['scene_params']['real_target_pos'],
    release_pos=res['bomb_release_pos'],
    explosion_pos=res['explosion_pos']
)
fig3 = plot_distance_curve(
    time_series=res['time_series'],
    min_distance=res['min_distance'],
    cover_status=res['cover_status'],
    effective_radius=res['scene_params']['effective_radius']
)
fig4 = plot_height_comparison(
    time_series=res['time_series'],
    missile_traj=res['missile_trajectory'],
    smoke_traj=res['smoke_trajectory']
)

# === 保存为 PNG ===

```



```

out_dir = Path("figs_png")
out_dir.mkdir(parents=True, exist_ok=True)
ts = datetime.now().strftime("%Y%m%d_%H%M%S")

paths = {
    "timeline": out_dir / f"timeline_{ts}.png",
    "xy_traj": out_dir / f"xy_trajectory_{ts}.png",
    "distance": out_dir / f"distance_curve_{ts}.png",
    "height": out_dir / f"height_compare_{ts}.png",
}

fig1.savefig(paths["timeline"], dpi=300, bbox_inches="tight")
fig2.savefig(paths["xy_traj"], dpi=300, bbox_inches="tight")
fig3.savefig(paths["distance"], dpi=300, bbox_inches="tight")
fig4.savefig(paths["height"], dpi=300, bbox_inches="tight")

print("\nPNG 已保存: ")
for k, p in paths.items():
    print(f" - {k}: {p.resolve()}")
plt.show()

```

P1_problem_plot.py

```

# -*- coding: utf-8 -*-
import numpy as np
import matplotlib
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['PingFang SC', 'SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False

# ----- 常量 -----
G = 9.8          # 重力
vm = 300.0       # 导弹速度
vs = 3.0         # 烟幕下沉
R = 10.0         # 云团半径
T = 20.0         # 有效时长

# ----- 点位 -----
a = np.array([0.0, 0.0, 0.0]) # 假目标
b = np.array([0.0, 200.0, 0.0]) # 真目标
m0 = np.array([20000.0, 0.0, 2000.0]) # 导弹起点
fy0 = np.array([17800.0, 0.0, 1800.0]) # 无人机起点

```

```

# 无人机速度朝假目标（只算 xy）
vfy = 120.0
dir_xy = a[:2] - fy0[:2]
dir_xy = dir_xy / np.linalg.norm(dir_xy)
vfy_vec = np.array([vfy * dir_xy[0], vfy * dir_xy[1], 0.0])
print("无人机速度向量:", vfy_vec)

# 时间
t_drop = 1.5
t_delay = 3.6
t_blast = t_drop + t_delay

# 投放点
drop_pos = fy0 + vfy_vec * t_drop
# 自由落体位移（z 方向）
z_drop = 0.5 * G * (t_delay ** 2)
# 起爆点（xy 继续随无人机）
blast_pos_xy = drop_pos + vfy_vec * t_delay
blast_pos = np.array([blast_pos_xy[0], blast_pos_xy[1], drop_pos[2] - z_drop])

print("投放点:", drop_pos)
print("起爆点:", blast_pos)

# 导弹朝假目标
m_dir = a - m0
m_dir = m_dir / np.linalg.norm(m_dir)
vm_vec = vm * m_dir
print("导弹方向:", m_dir, "速度向量:", vm_vec)
print("导弹起点:", m0)

# 到达假目标时间（距离/速度）
t_hit = np.linalg.norm(a - m0) / vm
print("导弹到达假目标时间: %.2f s" % t_hit)

# ----- 一些函数（简单点） -----
def m_pos(t):
    return m0 + vm_vec * t

def s_pos(t):
    # t 为起爆后的时间
    return np.array([blast_pos[0], blast_pos[1], blast_pos[2] - vs * t])

def is_between(pm, ps, pt):

```

```

# 看烟幕是否在导弹和目标之间（锐角）
v1 = ps - pm
v2 = pt - pm
d = np.linalg.norm(v1) * np.linalg.norm(v2)
if d == 0:
    return False
return np.dot(v1, v2) / d > 0

def dist_point_line(p, p0, u):
    # p 到过 p0 且方向 u(单位) 的直线距离
    ap = p - p0
    t = np.dot(ap, u)
    foot = p0 + t * u
    return np.linalg.norm(p - foot)

def seg_sphere_hit(p0, p1, c, r):
    # 线段 p0-p1 与球(c,r)是否相交
    d = p1 - p0
    f = p0 - c
    a2 = np.dot(d, d)
    b2 = 2.0 * np.dot(f, d)
    c2 = np.dot(f, f) - r * r
    if np.dot(f, f) <= r * r: return True
    if np.dot(p1 - c, p1 - c) <= r * r: return True
    D = b2 * b2 - 4 * a2 * c2
    if D < 0 or a2 == 0: return False
    sD = np.sqrt(D)
    t1 = (-b2 - sD) / (2 * a2)
    t2 = (-b2 + sD) / (2 * a2)
    return (0.0 <= t1 <= 1.0) or (0.0 <= t2 <= 1.0)

def pass_through(pm, ps, pm0, ps0, r):
    # 相对运动：把球看静止
    q0 = pm0 - ps0
    q1 = pm - ps
    return seg_sphere_hit(q0, q1, np.zeros(3), r)

def calc_time(dt=0.01):
    end_t = min(T, t_hit - t_blast)
    if end_t <= 0: return 0.0
    tot = 0.0
    pm0 = m_pos(t_blast)
    ps0 = s_pos(0.0)

```

```

t = 0.0
while t <= end_t + 1e-12:
    tt = t_blast + t
    if tt > t_hit: break
    pm = m_pos(tt)
    ps = s_pos(t)
    u = b - pm
    u = u / np.linalg.norm(u)
    dlos = dist_point_line(ps, pm, u)
    ok_between = is_between(pm, ps, b)
    in_ball = np.linalg.norm(pm - ps) <= R
    through = pass_through(pm, ps, pm0, ps0, R)
    if (dlos <= R and ok_between) or in_ball or through:
        tot += dt
    pm0, ps0 = pm, ps
    t += dt
return tot

eff = calc_time(0.01)

print("\n")
print("烟雾弹投放时间: %.1f s" % t_drop)
print("烟雾弹起爆时间: %.1f s" % t_blast)
print("起爆点位置: (%.1f, %.1f, %.1f)" % (blast_pos[0], blast_pos[1], blast_pos[2]))
print("导弹到达假目标时间: %.2f s" % t_hit)
print("有效遮蔽时长: %.3f 秒" % eff)

# ----- 一些时间点看看 -----
print("\n 关键时间点 ")
chk = [0, 1, 2, 3]
end_cap = min(T, t_hit - t_blast)
for t0 in chk:
    if t0 <= end_cap:
        tt = t_blast + t0
        pm = m_pos(tt)
        ps = s_pos(t0)
        u = b - pm
        u = u / np.linalg.norm(u)
        dlos = dist_point_line(ps, pm, u)
        dms = np.linalg.norm(pm - ps)
        v1 = ps - pm
        v2 = b - pm
        ang = np.degrees(np.arccos(

```

```

        np.clip(np.dot(v1, v2) / (np.linalg.norm(v1) * np.linalg.norm(v2)), -1, 1)
    ))
    ok_between = is_between(pm, ps, b)
    ok_eff = (dlos <= R and ok_between) or (dms <= R)
    print("起爆后 %ds:" % t0)
    print("  导弹:", pm)
    print("  云团:", ps)
    print("  到视线距离: %.2f m" % dlos)
    print("  直接距离: %.2f m" % dms)
    print("  在中间: ", "是" if ok_between else "否")
    print("  角度: %.1f°" % ang)
    print("  在云内: ", "是" if dms <= R else "否")
    print("  有效遮蔽(简): ", "是" if ok_eff else "否")

# ----- 画图 -----
fig = plt.figure(figsize=(15, 10))
ax = fig.add_subplot(111, projection='3d')

# 导弹轨迹
tt_m = np.linspace(0, t_hit, 200)
traj_m = np.array([m_pos(t) for t in tt_m])
ax.plot(traj_m[:,0], traj_m[:,1], traj_m[:,2], 'r-', lw=2, label='导弹轨迹')

# 烟幕轨迹（从起爆）
end_s = max(0.0, min(T, t_hit - t_blast))
tt_s = np.linspace(0, end_s, 100) if end_s > 0 else np.array([0.0])
traj_s = np.array([s_pos(t) for t in tt_s])
ax.plot(traj_s[:,0], traj_s[:,1], traj_s[:,2], 'b-', lw=2, label='烟幕下沉轨迹')

# 关键点
ax.scatter(*a, c='red', s=60, marker='x', label='假目标')
ax.scatter(*b, c='blue', s=60, marker='o', label='真目标')
ax.scatter(*m0, c='orange', s=40, label='导弹起点')
ax.scatter(*blast_pos, c='green', s=40, label='起爆点')

# 画几个球面（t=0/10/20）
for t in [0, 10, 20]:
    if 0 <= t <= end_s:
        c = s_pos(t)
        u = np.linspace(0, 2*np.pi, 40)
        v = np.linspace(0, np.pi, 20)
        x = c[0] + R * np.outer(np.cos(u), np.sin(v))
        y = c[1] + R * np.outer(np.sin(u), np.sin(v))

```

```

z = c[2] + R * np.outer(np.ones_like(u), np.cos(v))
ax.plot_surface(x, y, z, color='cyan', alpha=0.25, linewidth=0)

ax.view_init(elev=20, azim=45)
ax.set_xlabel('X (m)')
ax.set_ylabel('Y (m)')
ax.set_zlabel('Z (m)')
ax.set_title('烟幕遮蔽分析')
ax.legend()
ax.grid(True)
plt.tight_layout()
plt.show()

```

plotproblem1.py

```

# -*- coding: utf-8 -*-
import os
import numpy as np
import matplotlib.pyplot as plt
from datetime import datetime
from matplotlib.patches import Circle
import mpl_toolkits.mplot3d.art3d as art3d

plt.rcParams['font.sans-serif'] = ['PingFang SC', 'SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False

a = np.array([0.0, 0.0, 0.0])    # 假目标
b = np.array([0.0, 200.0, 0.0]) # 真目标

m = {
    'm1': np.array([20000.0, 0.0, 2000.0]),
    'm2': np.array([19000.0, 600.0, 2100.0]),
    'm3': np.array([18000.0, -600.0, 1900.0]),
}

d = { # 无人机
    'fy1': np.array([17800.0, 0.0, 1800.0]),
    'fy2': np.array([12000.0, 1400.0, 1400.0]),
    'fy3': np.array([6000.0, -3000.0, 700.0]),
    'fy4': np.array([11000.0, 2000.0, 1800.0]),
    'fy5': np.array([13000.0, -2000.0, 1300.0]),
}

def draw_cyl(ax, c, r, h, col='red', alp=0.9):

```

```

bot = Circle((c[0], c[1]), r, color=col, alpha=alp)
ax.add_patch(bot)
art3d.pathpatch_2d_to_3d(bot, z=c[2], zdir="z")
top = Circle((c[0], c[1]), r, color=col, alpha=alp)
ax.add_patch(top)
art3d.pathpatch_2d_to_3d(top, z=c[2] + h, zdir="z")

# 输出目录
out_dir = "battlefield_images"
if not os.path.exists(out_dir):
    os.makedirs(out_dir)
ts = datetime.now().strftime("%Y%m%d_%H%M%S")

# 画布
fig = plt.figure(figsize=(16, 8))

# ----- 左侧全景 -----
ax1 = fig.add_subplot(121, projection='3d')

draw_cyl(ax1, a, 10, 15, 'red', 0.9)
ax1.text(a[0], a[1], a[2] + 20, '假目标', color='red', fontsize=14, ha='center',
        bbox=dict(boxstyle="round,pad=0.3", facecolor="white", alpha=0.8))

draw_cyl(ax1, b, 10, 20, 'blue', 0.9)
ax1.text(b[0], b[1], b[2] + 25, '真目标', color='blue', fontsize=14, ha='center',
        bbox=dict(boxstyle="round,pad=0.3", facecolor="white", alpha=0.8))

# 点位
for k, p in m.items():
    ax1.scatter(p[0], p[1], p[2], c='red', marker='^', s=300, edgecolors='black', linewidth=2)
    ax1.text(p[0] + 500, p[1] + 500, p[2] + 500, k.upper(), fontsize=12,
            bbox=dict(boxstyle="round,pad=0.2", facecolor="white", alpha=0.7))

for k, p in d.items():
    ax1.scatter(p[0], p[1], p[2], c='green', marker='s', s=200, edgecolors='black', linewidth=1.5)
    ax1.text(p[0] + 500, p[1] + 500, p[2] + 500, k.upper(), fontsize=11,
            bbox=dict(boxstyle="round,pad=0.2", facecolor="white", alpha=0.7))

# 箭头（导弹指向假目标）
for p in m.values():
    v = a - p
    v = v / np.linalg.norm(v)
    L = 1000.0

```

```

ax1.quiver(p[0], p[1], p[2], v[0]*L, v[1]*L, v[2]*L,
           color='orange', arrow_length_ratio=0.25, linewidth=3, alpha=0.9)

ax1.set_xlabel('X (m)')
ax1.set_ylabel('Y (m)')
ax1.set_zlabel('Z (m)')
ax1.set_title('导弹/无人机分布（全景）')
ax1.set_xlim([-2000, 22000])
ax1.set_ylim([-4500, 4500])
ax1.set_zlim([-1000, 3000])
ax1.grid(True, alpha=0.4)
ax1.view_init(elev=20, azimuth=-60)

ax1.text2D(0.02, 0.95,
           "说明:\n红圆柱=假目标\n蓝圆柱=真目标\n红三角=导弹\n绿方块=无人机\n橙箭头=飞行方向",
           transform=ax1.transAxes, fontsize=10, va='top',
           bbox=dict(boxstyle="round,pad=0.4", facecolor="lightyellow", alpha=0.8))

ax2 = fig.add_subplot(122, projection='3d')
draw_cyl(ax2, a, 10, 15, 'red', 0.9)
draw_cyl(ax2, b, 10, 20, 'blue', 0.9)
ax2.text(a[0], a[1], a[2] + 18, '假目标', color='red', fontsize=13, ha='center')
ax2.text(b[0], b[1], b[2] + 23, '真目标', color='blue', fontsize=13, ha='center')

ax2.set_xlim([-50, 250])
ax2.set_ylim([-50, 250])
ax2.set_zlim([-10, 60])
ax2.set_xlabel('X (m)')
ax2.set_ylabel('Y (m)')
ax2.set_zlabel('Z (m)')
ax2.set_title('目标区特写')
ax2.grid(True, alpha=0.4)
ax2.view_init(elev=35, azimuth=-45)

ax2.plot([0, 0], [0, 200], [15, 20], 'k--', alpha=0.6)
ax2.text(10, 100, 18, '200m', fontsize=11,
        bbox=dict(boxstyle="round,pad=0.2", facecolor="white", alpha=0.7))

plt.tight_layout()

png_path = os.path.join(out_dir, f"battlefield_3d_map_{ts}.png")
pdf_path = os.path.join(out_dir, f"battlefield_3d_map_{ts}.pdf")
plt.savefig(png_path, dpi=300, bbox_inches='tight', facecolor='white')

```



```
plt.savefig(pdf_path, bbox_inches='tight', facecolor='white')

print(out_dir)
print("图片已经保存到", os.path.basename(png_path))
print("PDF 路径", os.path.basename(pdf_path))

plt.show()
```